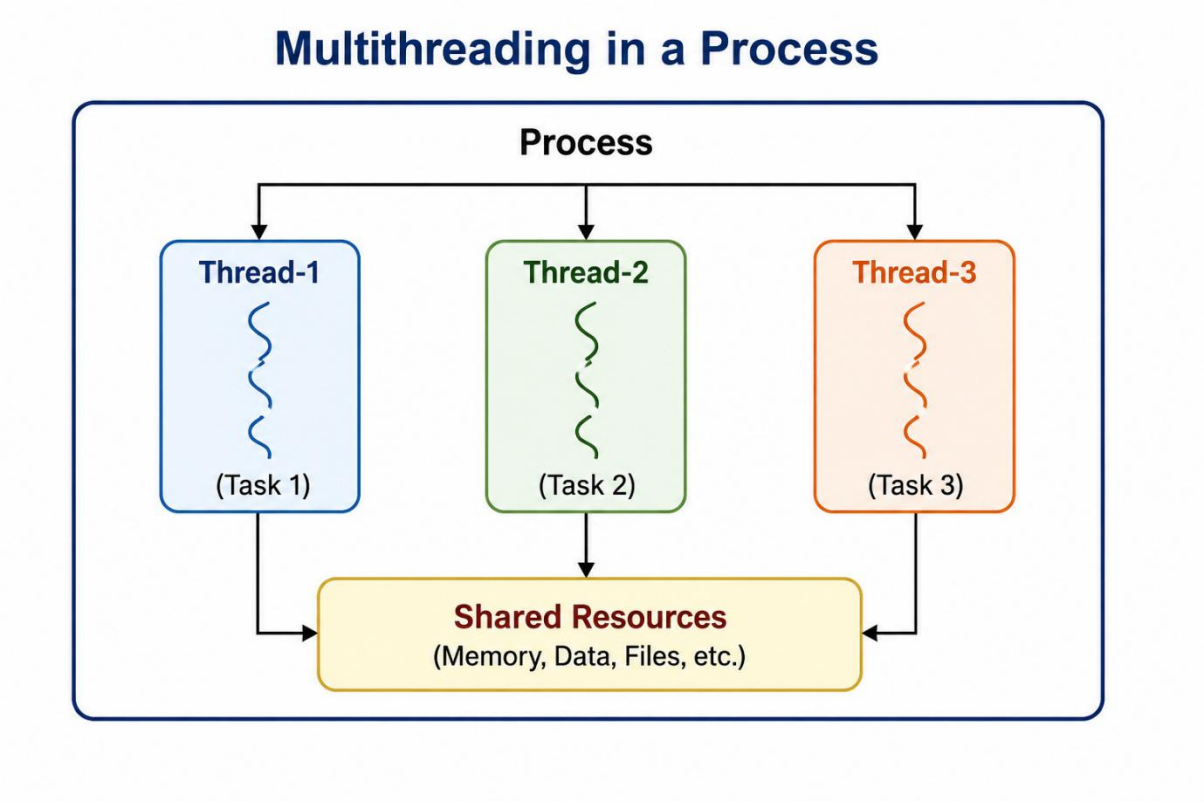

STUDY NOTES
ADVANCED
JAVA
Concepts • Technologies • Applications
Learn • Code • Excel
Java™
INSIDE THESE NOTES
Multithreading
Threads, Synchronization, Inter-Thread Communication
Collection Framework
Lists, Sets, Maps, Generics, Iterators
JDBC
Database Connectivity, CRUD Operations
Servlets
Web Components, Request Handling
Handling Form Data
GET, POST, Cookies, Sessions
JSP
JSP Elements, Implicit Objects, JavaBeans
import java.util.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class AdvancedJava {
public static void main(String[] args) {
List<String> list = new ArrayList<>();
list.add("Advanced Java");
list.add("Learn • Code • Excel");
for (String s : list) {
System.out.println(s);
}
}
}
Java
MULTITHREADING
JDBC
SERVLETS
JSP
Mr. D. R. Dhumal
EDUCATOR • MENTOR • GUIDE

Unit I: Multithreading

🟦 1.1 Introduction to Multithreading

- **Multithreading** is one of the most important features of Java. It allows a program to execute two or more threads concurrently within a single process. This enables multiple tasks to run at the same time and improves the overall performance and responsiveness of the application.
- A **thread** is the smallest unit of execution within a process. Each thread performs a specific task independently, and multiple threads within the same process share common memory and resources. This sharing makes communication between threads faster and more efficient.
- In a **single-threaded** program, only one task is executed at a time. If one task takes a long time to complete, the remaining tasks must wait until it finishes. This increases the execution time and reduces the responsiveness of the application.
- **Multithreading** overcomes this limitation by allowing multiple threads to execute concurrently. While one thread is waiting for an input or output operation, another thread can continue its execution. This improves CPU utilization, reduces idle time, and increases the overall efficiency of the application.
- Java provides **built-in support for multithreading**, making it easier to develop efficient and responsive applications. The **Java Virtual Machine (JVM)**, with the support of the operating system, manages the execution of multiple threads efficiently.
- Multithreading is widely used in modern software applications such as **web browsers, web servers, online banking systems, chat applications, games, media players, and file download managers**. It allows these applications to perform multiple tasks simultaneously and provides a better user experience.



🟦 1.2 Creating Threads

Java provides built-in support for creating and managing threads. A thread can be created in two ways: by extending the **Thread** class or by implementing the **Runnable** interface. Both methods allow multiple tasks to execute concurrently and improve the performance and responsiveness of an application. In modern Java applications, implementing the **Runnable** interface is generally preferred because it provides better flexibility and code reusability.

🟦 Creating Thread using Thread Class

A thread can be created by extending the **Thread** class. The programmer overrides the **run()** method and writes the task to be performed inside it. The **start()** method is then called to begin thread execution. When the **start()** method is invoked, a new thread is created, and the **run()** method is executed automatically.

Program

```
class MyThread extends Thread
{
   public void run()
   {
      System.out.println("Thread is running...");
   }

   public static void main(String args[])
   {
      MyThread t = new MyThread();
      t.start();
   }
}
```

Output

```
Thread is running...
```

🟦 Creating Thread using Runnable Interface

Another way to create a thread is by implementing the **Runnable** interface. The class implements the **run()** method, which contains the code to be executed by the thread. An object of the class is passed to the **Thread** class constructor, and the thread is started using the **start()** method. This approach is preferred because a class can extend another class while implementing the **Runnable** interface.

Program

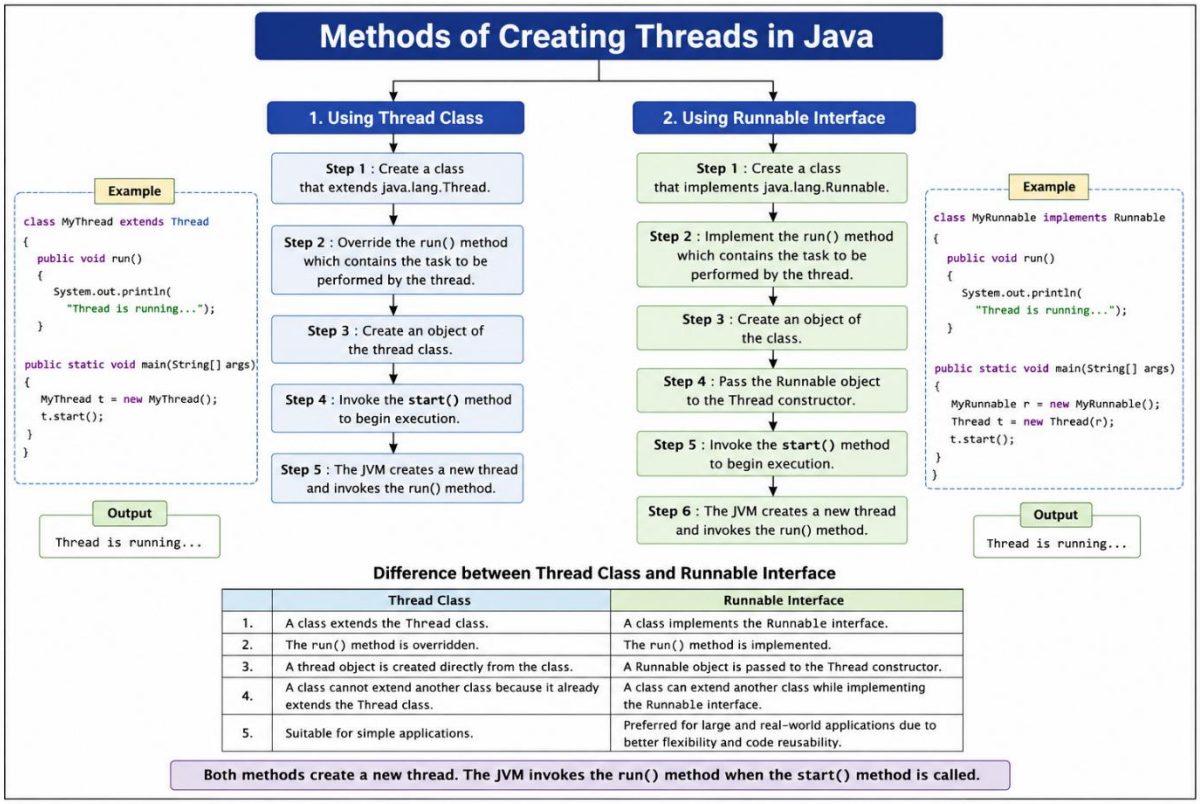
```
class MyRunnable implements Runnable
{
   public void run()
   {
      System.out.println("Thread is running...");
   }

   public static void main(String args[])
   {
      MyRunnable r = new MyRunnable();
      Thread t = new Thread(r);
      t.start();
   }
}
```

Output

```
Thread is running...
```

🟦 Difference between Thread Class and Runnable Interface

Thread Class	Runnable Interface
A class extends the **Thread** class.	A class implements the **Runnable** interface.
The **run()** method is overridden.	The **run()** method is implemented.
A thread object is created directly from the class.	A **Runnable** object is passed to the **Thread** constructor.
A class cannot extend another class because it already extends the **Thread** class.	A class can extend another class while implementing the **Runnable** interface.
Suitable for simple applications.	Preferred for large and real-world applications due to better flexibility and code reusability.



1.3 Thread Life Cycle

The **Thread Life Cycle** describes the different states through which a thread passes during its execution. A thread does not start executing immediately after it is created. It moves through different states such as **New, Runnable, Running, Blocked/Waiting, and Dead (Terminated)** until its execution is completed. The **Java Virtual Machine (JVM)**, with the support of the operating system, manages the movement of a thread from one state to another.

New State

A thread is in the **New** state when a thread object is created, but the **start()** method has not yet been called. At this stage, the thread is ready to start but is not eligible for CPU execution.

Runnable State

When the **start()** method is called, the thread enters the **Runnable** state. In this state, the thread is ready for execution and waits for the CPU scheduler to allocate CPU time. A thread may remain in this state until it is selected for execution.

Running State

A thread enters the **Running** state when the CPU scheduler selects it from the Runnable state. In this state, the **run()** method is executed, and the thread performs its assigned task. During execution, the thread may continue running, move to the Blocked/Waiting state, or complete its execution.

Blocked / Waiting State

A thread enters the **Blocked** or **Waiting** state when it is temporarily unable to continue execution. This may occur while waiting for user input, I/O operations, shared resources, another thread to complete, or methods such as **sleep()**, **wait()**, or **join()**. After the required condition is satisfied, the thread returns to the Runnable state.

🟦 Dead (Terminated) State

A thread enters the **Dead** or **Terminated** state after completing its execution or when it is terminated due to an error or an uncaught exception. Once a thread reaches this state, it cannot be restarted by calling the **start()** method again. To execute the task again, a new thread object must be created.

🟦 1.4 Thread Priorities

Every thread in Java is assigned a **priority** that indicates its relative importance during execution. The **Thread Scheduler** uses thread priority to decide which thread should be considered for execution. A thread with a higher priority is generally given preference over a thread with a lower priority. However, thread priority **does not guarantee** the execution order because the final scheduling decision depends on the **Java Virtual Machine (JVM)** and the **operating system**.

Java defines thread priorities in the range of **1 to 10**. By default, every newly created thread has a priority of **5**, which is called the normal priority. The priority of a thread can be changed using the **setPriority()** method and retrieved using the **getPriority()** method.

🟦 Thread Priority

Thread priority is an integer value that determines the relative importance of a thread. A higher priority thread is more likely to be selected for execution before a lower priority thread. However, the actual execution sequence is controlled by the JVM and the operating system scheduler.

🟦 MIN_PRIORITY

MIN_PRIORITY is the lowest priority that can be assigned to a thread. Its constant value is **1**. Threads with this priority are generally given the least preference during scheduling.

```
Thread.MIN_PRIORITY   // Value = 1
```

NORM_PRIORITY

NORM_PRIORITY is the default priority assigned to every thread. Its constant value is **5**. If no priority is explicitly assigned, the thread automatically receives this priority.

```
Thread.NORM_PRIORITY   // Value = 5
```

MAX_PRIORITY

MAX_PRIORITY is the highest priority that can be assigned to a thread. Its constant value is **10**. Threads with this priority are generally given higher preference by the thread scheduler.

```
Thread.MAX_PRIORITY   // Value = 10
```

---

getPriority()

The **getPriority()** method is used to retrieve the current priority of a thread. It returns an integer value between **1** and **10**.

Syntax

```
int p = thread.getPriority();
```

---

setPriority()

The **setPriority()** method is used to assign a new priority to a thread. The priority value must be between **1** and **10**. If an invalid priority value is specified, Java throws an **IllegalArgumentException**.

Syntax

```
thread.setPriority(priority);
```

Program

```
class PriorityDemo extends Thread
{
   public void run()
   {
      System.out.println(Thread.currentThread().getName() +
            " Priority : " + Thread.currentThread().getPriority());
   }

   public static void main(String args[])
   {
      PriorityDemo t1 = new PriorityDemo();
      PriorityDemo t2 = new PriorityDemo();
      PriorityDemo t3 = new PriorityDemo();

      t1.setName("Thread-1");
      t2.setName("Thread-2");
      t3.setName("Thread-3");

      t1.setPriority(Thread.MIN_PRIORITY);
      t2.setPriority(Thread.NORM_PRIORITY);
      t3.setPriority(Thread.MAX_PRIORITY);

      t1.start();
      t2.start();
      t3.start();
```

```
    }
}
```

Output

Thread-1 Priority : 1

Thread-2 Priority : 5

Thread-3 Priority : 10

Note: Thread priority only indicates the relative importance of a thread. It does not guarantee that a higher priority thread will always execute before a lower priority thread, because thread scheduling depends on the JVM and the operating system.

🟦 1.5 Thread Synchronization

In a multithreaded environment, multiple threads may access the same resource simultaneously. If two or more threads try to read or modify shared data at the same time, the result may become incorrect or inconsistent. This situation is known as a **race condition**. To avoid such problems, Java provides **Thread Synchronization**.

Thread Synchronization is a mechanism that controls the access of multiple threads to a shared resource. It ensures that only one thread can access the critical section of the code at a time, while the other threads wait until the resource becomes available. This helps maintain data consistency, prevents conflicts between threads, and ensures the correct execution of a multithreaded program.

🟦 Need of Synchronization

Synchronization is required whenever multiple threads access the same resource simultaneously. Without synchronization, the output of the program may become unpredictable because different threads may interfere with each other's execution. Synchronization ensures that shared resources are accessed in a controlled manner and only one thread performs the critical operation at a time.

The main need for synchronization is to:

- Prevent race conditions.
- Maintain data consistency.
- Protect shared resources from concurrent access.
- Ensure thread-safe execution.
- Improve the reliability of multithreaded applications.

🟦 synchronized Method

A **synchronized method** allows only one thread to execute the method at a time. When a thread enters a synchronized method, it acquires the lock associated with the object. Other threads attempting to execute the same synchronized method on the same object must wait until the lock is released. This prevents multiple threads from modifying shared data simultaneously and ensures safe execution.

Program

```
class Table
{
   synchronized void printTable(int n)
   {
      for(int i=1; i<=5; i++)
      {
         System.out.println(n*i);
         try
         {
            Thread.sleep(500);
         }
         catch(Exception e) {}
      }
```

```
    }
}

class MyThread1 extends Thread
{
    Table t;

    MyThread1(Table t)
    {
        this.t = t;
    }

    public void run()
    {
        t.printTable(5);
    }
}

class MyThread2 extends Thread
{
    Table t;

    MyThread2(Table t)
    {
        this.t = t;
    }
```

```
    public void run()
    {
        t.printTable(10);
    }
}

public class SynchronizationDemo
{
    public static void main(String args[])
    {
        Table obj = new Table();

        MyThread1 t1 = new MyThread1(obj);
        MyThread2 t2 = new MyThread2(obj);

        t1.start();
        t2.start();
    }
}
```

Output

5

10

15

20

25

10

20

30

40

50

🟦 synchronized Block

Sometimes, only a small portion of a method requires synchronization. In such cases, Java provides a **synchronized block**, which locks only the critical section instead of the entire method. This improves the performance of the application because the remaining code can be executed concurrently by other threads.

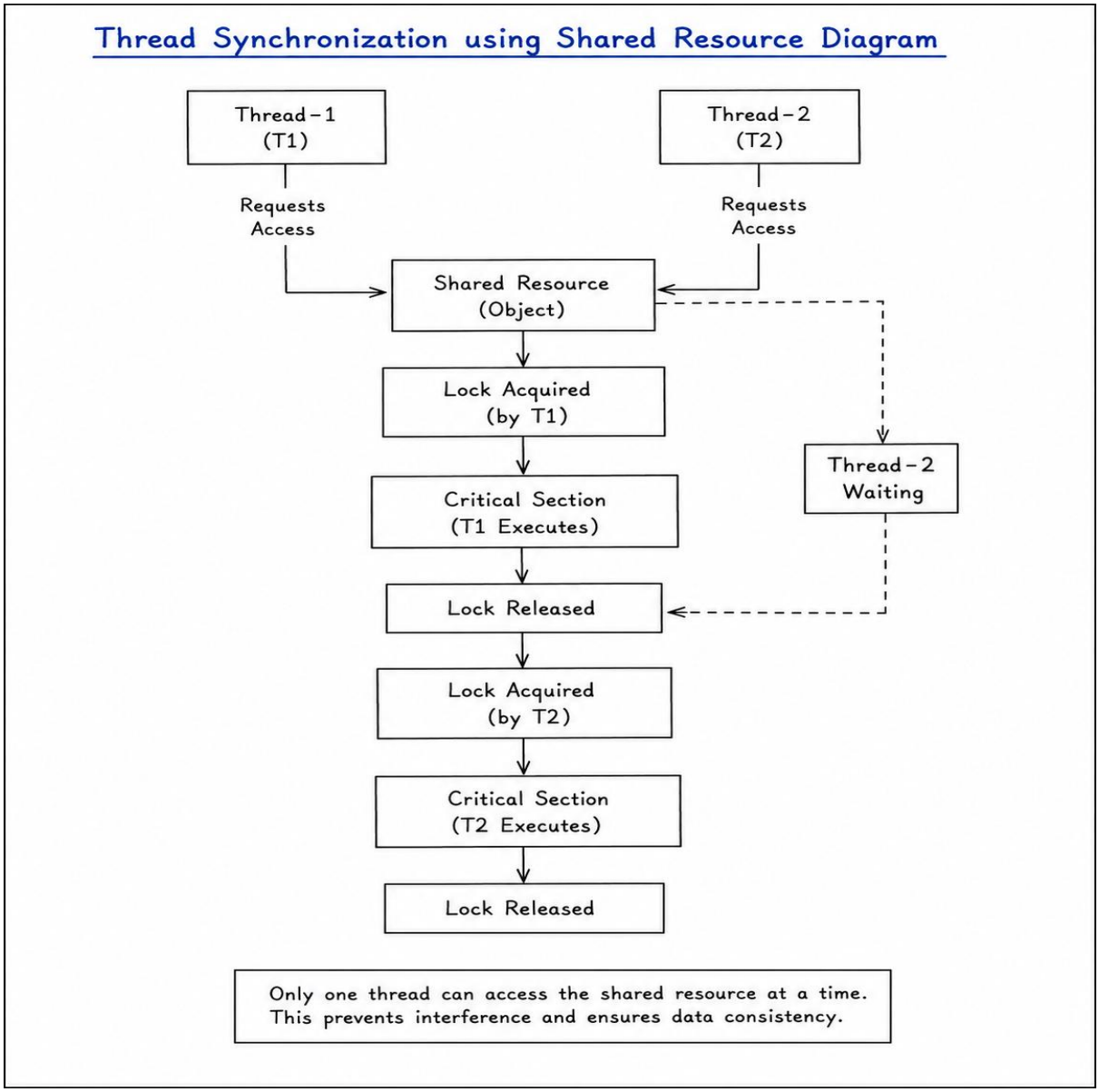
Syntax

```
synchronized(object)
{
   // Critical Section
}
```

A synchronized block is preferred when only a specific part of a method accesses shared resources.

Advantages of Synchronization

- Prevents race conditions.
- Maintains data consistency and integrity.
- Ensures thread-safe access to shared resources.
- Prevents simultaneous modification of shared data.
- Improves the reliability and correctness of multithreaded applications.



🟦 Most Important Questions (7–8 Marks)

Q.1 Explain **Multithreading** in Java. Describe the **Thread Life Cycle** with a neat diagram.

Q.2 Explain the different ways of **creating threads** in Java. Compare the **Thread Class** and **Runnable Interface** with suitable programs.

Q.3 What is **Thread Synchronization**? Explain the **need of synchronization**. Describe **synchronized method** and **synchronized block** with a suitable program.

Q.4 What are **Thread Priorities**? Explain **MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY, getPriority()** and **setPriority()** with a suitable program.

Q.5 Write short notes on any **TWO** of the following:

- Introduction to Multithreading
- Thread Life Cycle
- Thread Priorities
- Thread Synchronization
- Creating Threads

Unit II: Collection Framework

🟦 2.1 Collection Framework

The **Collection Framework** in Java is a unified architecture that provides a set of interfaces and classes to store, manage, and manipulate groups of objects efficiently. It simplifies data handling by providing ready-made data structures and algorithms. Instead of creating custom data structures, programmers can use the Collection Framework to perform operations such as storing, searching, inserting, deleting, sorting, and traversing data.

The Java Collection Framework is available in the **java.util** package and provides reusable interfaces, classes, and algorithms that improve programming efficiency and code reusability. Collections can grow or shrink dynamically as elements are added or removed, eliminating the need to specify the size in advance.

🟦 Features of Collection Framework

- Provides a standard architecture for storing and managing collections of objects.
- Supports dynamic storage without specifying the size in advance.
- Offers built-in methods for insertion, deletion, searching, sorting, and updating data.
- Improves code reusability and programming efficiency.
- Reduces programming effort by providing ready-made data structures and algorithms.
- Supports type-safe programming through **Generics**.
- Provides high-performance data structures for different application requirements.

🟦 Components of Collection Framework

The Java Collection Framework mainly consists of the following components:

1. Interfaces

Interfaces define the basic operations that can be performed on collections. Some important interfaces are:

- Collection
- List
- Set
- Queue
- Map *(Map is a part of the Collection Framework but does not extend the Collection interface.)*

---

2. Classes

Classes provide the implementation of collection interfaces. Different classes are designed for different data storage and retrieval requirements. Some commonly used classes are:

- ArrayList
- Vector
- LinkedList
- HashSet
- TreeSet
- HashMap
- Hashtable
- TreeMap

---

3. Algorithms

The Collection Framework provides built-in algorithms through the **Collections** class for performing common operations such as:

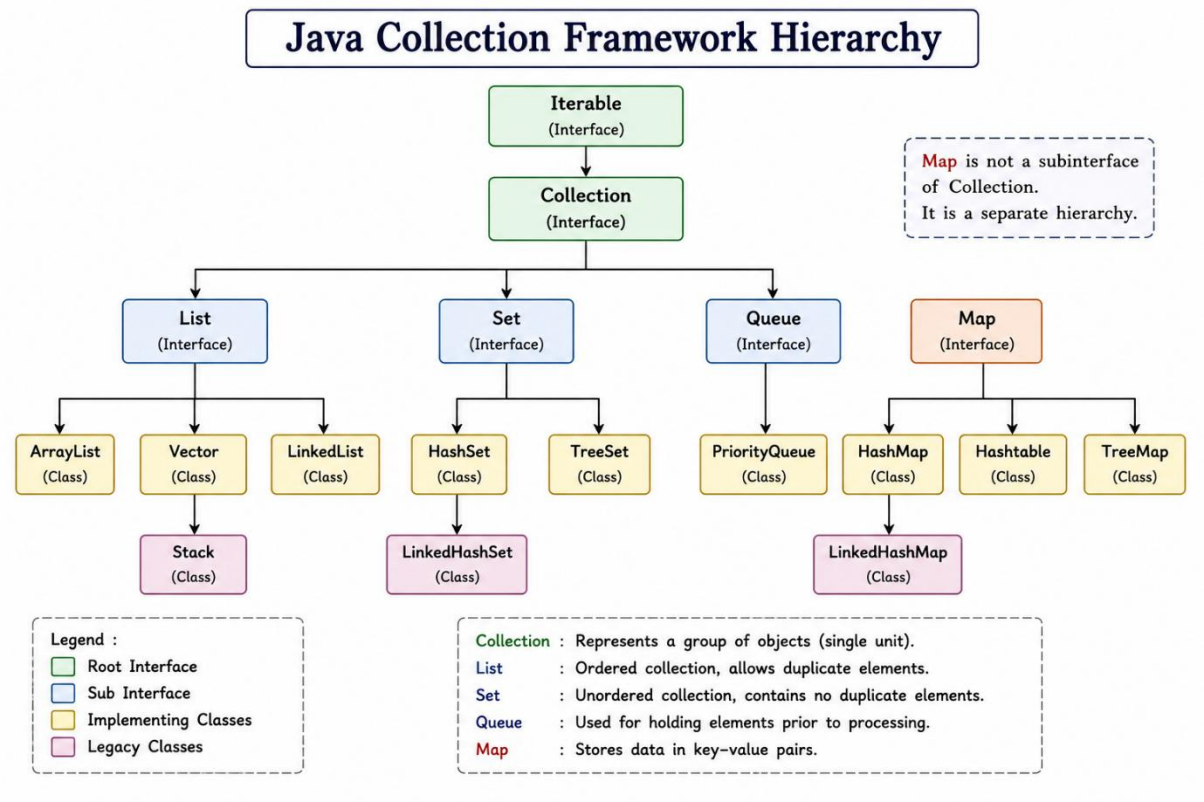
- Sorting
- Searching
- Binary Search
- Reversing
- Shuffling
- Finding maximum and minimum elements

🟦 Advantages of Collection Framework

- Simplifies data management.
- Reduces programming effort.
- Improves code reusability.
- Provides reusable and efficient data structures.
- Supports dynamic storage of elements.
- Improves application performance.
- Makes code easier to maintain and understand.
- Provides built-in algorithms for common operations.

🟦 Hierarchy of Collection Framework

The Collection Framework consists of interfaces and their implementing classes. The main hierarchy starts with the **Collection** interface, which is extended by the **List**, **Set**, and **Queue** interfaces. The **Map** interface is a separate part of the Collection Framework and does not inherit from the **Collection** interface. Each interface provides different implementations based on application requirements.



2.2 Collection Interface

The **Collection** interface is the root interface of the Java Collection Framework. It is available in the **java.util** package and represents a group of objects, known as elements. The Collection interface defines the basic operations that can be performed on collections. It serves as the foundation for other interfaces such as **List**, **Set**, and **Queue**. However, the **Map** interface is a separate part of the Collection Framework and does not extend the Collection interface.

The Collection interface does not provide a direct implementation. It is indirectly implemented by various collection classes through its subinterfaces such as **List**, **Set**, and **Queue**. These classes provide different ways of storing and managing data according to application requirements.

Features of Collection Interface

- Represents a group of objects as a single unit.
- Provides a common set of methods for all collection classes.
- Supports dynamic storage of elements.
- Allows easy insertion, deletion, searching, and traversal of elements.
- Supports different types of collections through **List**, **Set**, and **Queue** interfaces.
- Improves code reusability and flexibility.
- Supports type-safe programming using **Generics**.

Common Methods of Collection Interface

Method	Description
add(E e)	Adds an element to the collection.
addAll(Collection c)	Adds all elements of another collection.
remove(Object o)	Removes the specified element from the collection.
contains(Object o)	Checks whether the specified element is present in the collection.
size()	Returns the total number of elements in the collection.
isEmpty()	Checks whether the collection is empty.
clear()	Removes all elements from the collection.
iterator()	Returns an Iterator object to traverse the collection.
toArray()	Converts the collection into an array.

Program

```
import java.util.*;

public class CollectionDemo
{
   public static void main(String args[])
   {
      Collection<String> c = new ArrayList<String>();

      c.add("Java");
      c.add("Python");
      c.add("C++");

      System.out.println("Collection : " + c);
      System.out.println("Size : " + c.size());
      System.out.println("Contains Java : " + c.contains("Java"));

      c.remove("Python");

      System.out.println("After Removal : " + c);

      c.clear();

      System.out.println("Is Collection Empty? " + c.isEmpty());
   }
}
```

Output

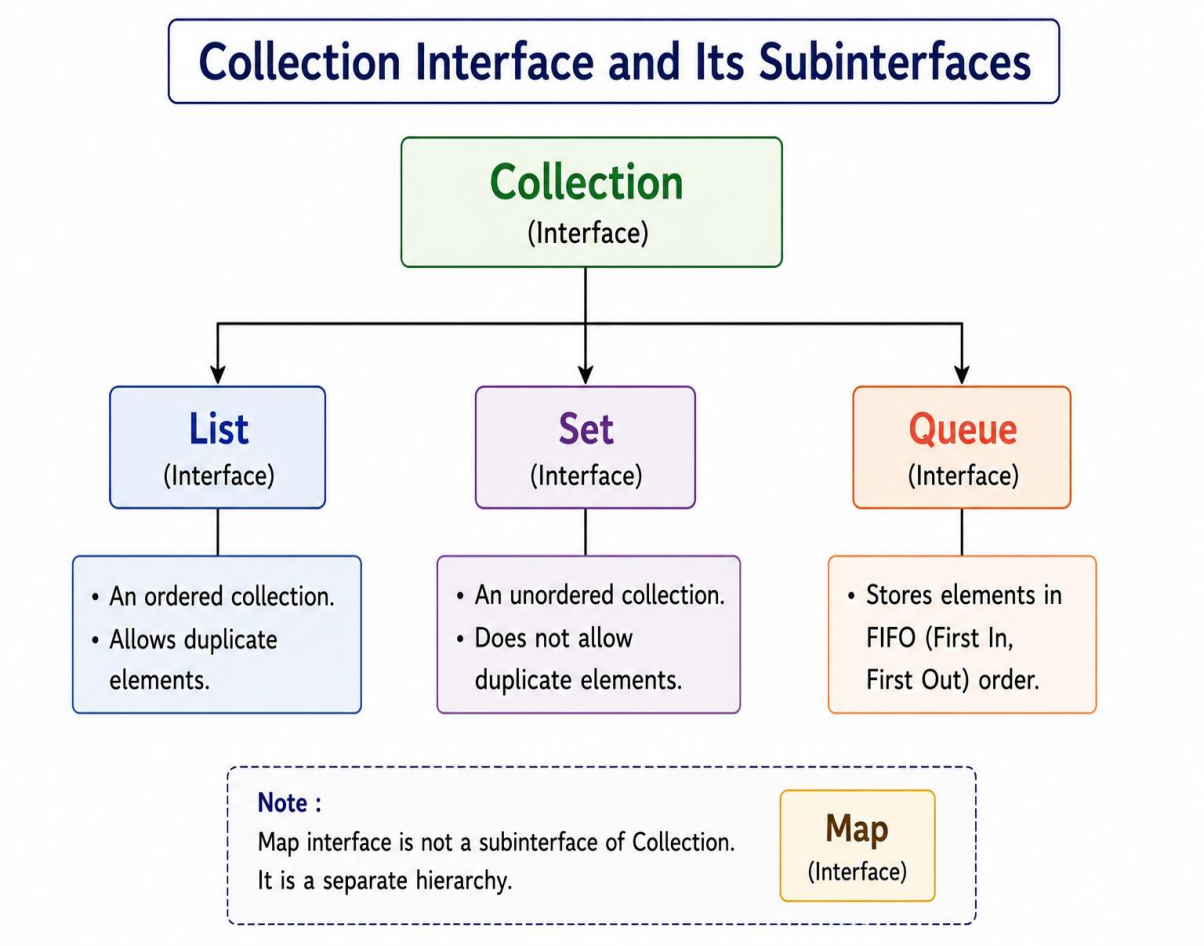
Collection : [Java, Python, C++]

Size : 3

Contains Java : true

After Removal : [Java, C++]

Is Collection Empty? True



🟦 2.3 ArrayList

ArrayList is a class of the Java Collection Framework that implements the **List** interface. It is available in the **java.util** package. ArrayList is a **resizable array implementation** of the List interface and is used to store a dynamic collection of elements. Unlike arrays, the size of an ArrayList automatically increases or decreases as elements are added or removed.

ArrayList preserves the **insertion order** of elements and allows **duplicate elements**. It also supports **random access**, which enables elements to be accessed quickly using their index. Since ArrayList stores elements internally using a dynamic array, insertion or deletion in the middle of the list may require shifting of elements, which affects performance.

🟦 Features of ArrayList

- Implements the **List** interface.
- Preserves the insertion order of elements.
- Allows duplicate elements.
- Supports automatic resizing.
- Allows random access using index values.
- Stores elements in a dynamic array.
- Does not provide synchronization.
- Allows multiple **null** values.

🟦 Constructors of ArrayList

ArrayList<E> list = new ArrayList<E>();

Creates an empty ArrayList.

ArrayList<E> list = new ArrayList<E>(int capacity);

Creates an ArrayList with the specified initial capacity.

ArrayList<E> list = new ArrayList<E>(Collection<? extends E> c);

Creates an ArrayList containing the elements of the specified collection.

🟦 Common Methods of ArrayList

Method	Description
add(E e)	Adds an element to the end of the list.
add(int index, E e)	Inserts an element at the specified index.
get(int index)	Returns the element at the specified index.
set(int index, E e)	Replaces the element at the specified index.
remove(int index)	Removes the element at the specified index.
contains(Object o)	Checks whether the specified element exists.
indexOf(Object o)	Returns the index of the first occurrence of the specified element.
lastIndexOf(Object o)	Returns the index of the last occurrence of the specified element.
size()	Returns the number of elements in the list.
isEmpty()	Checks whether the list is empty.
clear()	Removes all elements from the list.

🟦 Advantages of ArrayList

- Easy to use and implement.
- Automatically resizes as elements are added or removed.
- Provides fast random access to elements.
- Preserves the insertion order.
- Allows duplicate and **null** elements.
- Suitable for frequent retrieval operations.

🟦 Limitations of ArrayList

- Insertion and deletion in the middle of the list are slower because elements need to be shifted.
- It is not synchronized, so it is not thread-safe.
- Performance decreases when frequent insertions and deletions are required.

🟦 Program

```
import java.util.*;


public class ArrayListDemo
{
   public static void main(String args[])
   {
      ArrayList<String> list = new ArrayList<String>();


      list.add("Java");
      list.add("Python");
```

```
        list.add("C++");

        System.out.println("ArrayList : " + list);

        list.add(1, "HTML");
        System.out.println("After Insert : " + list);

        list.remove("Python");
        System.out.println("After Removal : " + list);

        System.out.println("Element at Index 1 : " + list.get(1));
        System.out.println("Size : " + list.size());
        System.out.println("Contains Java : " + list.contains("Java"));
    }
}
```

Output

```
ArrayList : [Java, Python, C++]
After Insert : [Java, HTML, Python, C++]
After Removal : [Java, HTML, C++]
Element at Index 1 : HTML
Size : 3
Contains Java : true
```

2.4 Vector

Vector is a class of the Java Collection Framework that implements the **List** interface. It is available in the **java.util** package. Vector is a **resizable array implementation** of the List interface and is used to store a dynamic collection of elements. Unlike arrays, its size automatically increases or decreases as elements are added or removed.

Vector preserves the **insertion order** of elements and allows **duplicate** as well as **null** elements. It provides **random access** to elements using index values. The main feature of Vector is that it is **synchronized**, which makes it **thread-safe**. Because of synchronization, Vector is generally slower than **ArrayList** and is mainly used in multi-threaded applications.

Features of Vector

- Implements the **List** interface.
- Preserves the insertion order of elements.
- Allows duplicate elements.
- Allows multiple **null** values.
- Supports automatic resizing.
- Provides random access using index values.
- Is synchronized and thread-safe.
- Supports legacy methods such as **addElement()** and **removeElement()**.
- Suitable for multi-threaded applications.

Constructors of Vector

Vector<E> v = new Vector<E>();

Creates an empty Vector.

Vector<E> v = new Vector<E>(int capacity);

Creates a Vector with the specified initial capacity.

Vector<E> v = new Vector<E>(int capacity, int capacityIncrement);

Creates a Vector with the specified initial capacity and capacity increment.

Vector<E> v = new Vector<E>(Collection<? extends E> c);

Creates a Vector containing the elements of the specified collection.

Common Methods of Vector

Method	Description
add(E e)	Adds an element to the end of the Vector.
add(int index, E e)	Inserts an element at the specified index.
get(int index)	Returns the element at the specified index.
set(int index, E e)	Replaces the element at the specified index.
remove(int index)	Removes the element at the specified index.
contains(Object o)	Checks whether the specified element exists.
firstElement()	Returns the first element of the Vector.
lastElement()	Returns the last element of the Vector.
size()	Returns the number of elements in the Vector.
capacity()	Returns the current capacity of the Vector.
isEmpty()	Checks whether the Vector is empty.
clear()	Removes all elements from the Vector.

🟦 Advantages of Vector

- Automatically resizes as elements are added or removed.
- Preserves the insertion order of elements.
- Provides fast random access to elements.
- Is synchronized and thread-safe.
- Allows duplicate and **null** elements.
- Suitable for multi-threaded applications.

🟦 Limitations of Vector

- Slower than **ArrayList** because of synchronization.
- It is a **legacy class**, and **ArrayList** is preferred for new applications.
- Not suitable for high-performance single-threaded applications.

🟦 Program

```
import java.util.*;


public class VectorDemo
{
   public static void main(String args[])
   {
      Vector<String> v = new Vector<String>();


      v.add("Java");
      v.add("Python");
      v.add("C++");
```

```
        System.out.println("Vector : " + v);

        v.add(1, "HTML");
        System.out.println("After Insert : " + v);

        v.remove("Python");
        System.out.println("After Removal : " + v);

        System.out.println("Element at Index 1 : " + v.get(1));
        System.out.println("Size : " + v.size());
        System.out.println("Capacity : " + v.capacity());
        System.out.println("Is Vector Empty? " + v.isEmpty());
    }
}
```

Output

```
Vector : [Java, Python, C++]
After Insert : [Java, HTML, Python, C++]
After Removal : [Java, HTML, C++]
Element at Index 1 : HTML
Size : 3
Capacity : 10
Is Vector Empty? false
```

2.5 Generics

Generics is a feature introduced in Java to provide **type safety** and improve code reusability. It allows programmers to create classes, interfaces, and methods that can work with different data types while ensuring compile-time type checking. Generics reduce the need for explicit type casting and minimize the chances of runtime errors.

Before Generics were introduced, collections could store objects of any type, which often resulted in **ClassCastException** at runtime. By using Generics, the compiler ensures that only objects of the specified type can be stored in a collection.

Features of Generics

- Provides compile-time type checking.
- Ensures type safety.
- Reduces explicit type casting.
- Minimizes runtime errors.
- Improves code reusability.
- Supports generic classes, generic interfaces, and generic methods.
- Makes programs easier to read and maintain.

Syntax of Generics

ClassName<DataType> object = new ClassName<DataType>();

Example

ArrayList<String> list = new ArrayList<String>();

🟦 Advantages of Generics

- Prevents **ClassCastException** at runtime.
- Detects type-related errors during compilation.
- Improves compile-time error checking.
- Improves code readability and maintainability.
- Increases code reusability.
- Makes collection classes type-safe.
- Reduces unnecessary type casting.
- Makes collection handling more secure and efficient.

🟦 Program

```
import java.util.*;


public class GenericsDemo
{
   public static void main(String args[])
   {
      ArrayList<Integer> list = new ArrayList<Integer>();


      list.add(10);
      list.add(20);
      list.add(30);


      for(Integer num : list)
      {
         System.out.println(num);
```

```
        }
    }
}
```

Output

10

20

30

🟦 2.6 Iterator

An **Iterator** is an interface in the Java Collection Framework that is used to **traverse** or **iterate** through the elements of a collection one by one. It is available in the **java.util** package. Iterator provides a common way to access elements of different collection classes such as **ArrayList**, **Vector**, **HashSet**, and **LinkedList** without knowing their internal implementation.

An Iterator object is obtained by calling the **iterator()** method of the Collection interface. It supports **forward traversal** only and allows elements to be removed safely from a collection during iteration using the **remove()** method.

🟦 Features of Iterator

- Used to traverse collection elements one by one.
- Supports only forward traversal.
- Does not support backward traversal.
- Works with most collection classes.
- Provides a uniform way to access collection elements.
- Allows safe removal of elements during iteration.
- Can traverse only one collection at a time.
- Improves code readability and flexibility.

🟦 Common Methods of Iterator

Method	Description
hasNext()	Returns **true** if the collection has more elements.
next()	Returns the next element in the collection.
remove()	Removes the current element from the collection.

🟦 Advantages of Iterator

- Provides a standard way to traverse collections.
- Eliminates the need for index-based traversal.
- Allows safe removal of elements while iterating.
- Works with different collection classes.
- Reduces the possibility of programming errors during traversal.
- Improves code reusability and maintainability.

🟦 Program

```
import java.util.*;


public class IteratorDemo
{
   public static void main(String args[])
   {
      ArrayList<String> list = new ArrayList<String>();


      list.add("Java");
      list.add("Python");
      list.add("C++");
```

```
        Iterator<String> itr = list.iterator();

        while(itr.hasNext())
        {
            String language = itr.next();

            if(language.equals("Python"))
            {
                itr.remove();
            }
        }

        System.out.println(list);
    }
}
```

Output

[Java, C++]

2.7 Comparable

The **Comparable** interface is used to define the **natural ordering** of objects in Java. It is available in the **java.lang** package. A class implements the Comparable interface when its objects need to be sorted based on a single field such as **ID, Name, Age, or Marks**. Comparable is mainly used when objects need to be sorted according to their natural ordering.

The Comparable interface contains a single method, **compareTo()**, which is used to compare one object with another. The **Collections.sort()** method uses the Comparable interface to sort objects in ascending order.

Features of Comparable

- Available in the **java.lang** package.
- Defines the natural ordering of objects.
- Supports sorting based on a single field.
- Supports ascending order sorting by default.
- Contains only one method, **compareTo()**.
- Used with **Collections.sort()** for sorting.
- Provides compile-time type safety using Generics.
- Suitable for sorting user-defined objects.

---

compareTo() Method

Syntax

public int compareTo(T obj)

Return Values	
Return Value	**Meaning**
< 0	Current object is smaller than the specified object.
0	Both objects are equal.
> 0	Current object is greater than the specified object.

🟦 Advantages of Comparable

- Provides natural ordering of objects.
- Easy to implement and use.
- Supports sorting of user-defined objects.
- Works efficiently with the **Collections.sort()** method.
- Required by many Java APIs that perform sorting.
- Improves code readability and reusability.

🟦 Program

```
import java.util.*;
class Student implements Comparable<Student>
{
   int id;
   String name;
   Student(int id, String name)
   {
      this.id = id;
      this.name = name;
   }
```

```
    public int compareTo(Student s)
    {
        return Integer.compare(this.id, s.id);
    }
}
public class ComparableDemo
{
    public static void main(String args[])
    {
        ArrayList<Student> list = new ArrayList<Student>();
        list.add(new Student(103, "Rahul"));
        list.add(new Student(101, "Amit"));
        list.add(new Student(102, "Priya"));

        Collections.sort(list);

        for(Student s : list)
        {
            System.out.println(s.id + " " + s.name);
        }
    }
}
```

Output

101 Amit

102 Priya

103 Rahul

🟦 2.8 TreeSet

TreeSet is a class of the Java Collection Framework that implements the **NavigableSet** interface, which extends the **SortedSet** interface, and indirectly implements the **Set** interface. It is available in the **java.util** package. TreeSet is used to store **unique elements** in **natural (ascending) order**. It automatically sorts the elements according to their natural ordering and does not allow duplicate elements.

TreeSet internally uses a **Red-Black Tree** data structure, which provides efficient searching, insertion, and deletion operations. It does not preserve the insertion order, does not support index-based access, and does not allow **null** elements.

🟦 Features of TreeSet

- Implements the **NavigableSet** interface.
- Stores unique elements only.
- Automatically sorts elements in natural (ascending) order.
- Does not allow duplicate elements.
- Does not preserve insertion order.
- Does not support index-based access.
- Uses the **Red-Black Tree** data structure.
- Does not allow **null** elements.
- Provides efficient searching, insertion, and deletion operations.

🟦 Common Methods of TreeSet

Method	Description
add(E e)	Adds an element to the TreeSet.
remove(Object o)	Removes the specified element.
contains(Object o)	Checks whether the specified element exists.
first()	Returns the first (lowest) element.
last()	Returns the last (highest) element.
higher(E e)	Returns the least element greater than the specified element.
lower(E e)	Returns the greatest element less than the specified element.
size()	Returns the number of elements.
isEmpty()	Checks whether the TreeSet is empty.
clear()	Removes all elements from the TreeSet.

🟦 Advantages of TreeSet

- Automatically stores elements in sorted order.
- Does not allow duplicate elements.
- Provides efficient searching and retrieval.
- Suitable for storing unique sorted data.
- Supports navigation methods such as **first()**, **last()**, **higher()**, and **lower()**.
- Suitable for applications requiring sorted and unique data.

🟦 Limitations of TreeSet

- Does not preserve insertion order.
- Does not support index-based access.
- Does not allow **null** elements.
- Does not allow heterogeneous objects.
- Slower than **HashSet** because elements are maintained in sorted order.
- Requires elements to be mutually comparable for sorting.

🟦 Program

```
import java.util.*;


public class TreeSetDemo
{
   public static void main(String args[])
   {
      TreeSet<String> set = new TreeSet<String>();


      set.add("Java");
      set.add("Python");
      set.add("C++");
      set.add("Java");


      System.out.println("TreeSet : " + set);
      System.out.println("First Element : " + set.first());
      System.out.println("Last Element : " + set.last());
      System.out.println("Contains Java : " + set.contains("Java"));
```

```
        System.out.println("Size : " + set.size());
        System.out.println("Is TreeSet Empty? " + set.isEmpty());

        set.remove("Java");
        System.out.println("After Removing Java : " + set);
    }
}
```

Output

```
TreeSet : [C++, Java, Python]
First Element : C++
Last Element : Python
Contains Java : true
Size : 3
Is TreeSet Empty? false
After Removing Java : [C++, Python]
```

🟦 2.9 HashSet

HashSet is a class of the Java Collection Framework that implements the **Set** interface. It is available in the **java.util** package. HashSet is used to store **unique elements** and does not allow duplicate values. Unlike TreeSet, HashSet does not maintain the insertion order or sorting of elements. The elements are stored based on their **hash code**, which provides fast insertion, deletion, and searching operations.

HashSet is internally **backed by a HashMap**. It allows only one **null** element and is best suited for applications where uniqueness of elements is required and the order of elements is not important.

🟦 Features of HashSet

- Implements the **Set** interface.
- Stores unique elements only.
- Does not allow duplicate elements.
- Stores elements based on their **hash code**.
- Does not maintain any predictable order of elements.
- Does not sort elements.
- Is internally backed by a **HashMap**.
- Allows only one **null** element.
- Provides fast insertion, deletion, and searching operations.
- Does not support index-based access.

Common Methods of HashSet	
Method	**Description**
add(E e)	Adds an element to the HashSet.
remove(Object o)	Removes the specified element.
contains(Object o)	Checks whether the specified element exists.
iterator()	Returns an Iterator to traverse the HashSet.
size()	Returns the number of elements.
isEmpty()	Checks whether the HashSet is empty.
clear()	Removes all elements from the HashSet.

Advantages of HashSet

- Stores only unique elements.
- Provides fast insertion, deletion, and searching operations.
- Provides constant-time performance for basic operations under normal conditions.
- Allows one **null** element.
- Suitable for applications where duplicate elements are not allowed.
- Offers better performance than **TreeSet** for basic operations.
- Easy to use and efficient for storing unordered data.

Limitations of HashSet

- Does not maintain the insertion order.
- Does not support sorting of elements.
- Does not support index-based access.
- Not synchronized, so it is not thread-safe.
- Cannot store duplicate elements.

Program

```
import java.util.*;

public class HashSetDemo
{
   public static void main(String args[])
   {
      HashSet<String> set = new HashSet<String>();

      set.add("Java");
      set.add("Python");
      set.add("C++");
      set.add("Java");

      System.out.println("HashSet : " + set);
      System.out.println("Contains Java : " + set.contains("Java"));
      System.out.println("Size : " + set.size());
      System.out.println("Is HashSet Empty? " + set.isEmpty());

      set.remove("Python");
      System.out.println("After Removing Python : " + set);

      System.out.println("Iterator Output :");

      Iterator<String> itr = set.iterator();
```

```
        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }
    }
}
```

Output

HashSet : [Java, Python, C++]

Contains Java : true

Size : 3

Is HashSet Empty? false

After Removing Python : [Java, C++]

Iterator Output :

Java

C++

Note: The order of elements displayed in a **HashSet** may vary because HashSet does **not** maintain any predictable order. Therefore, the output order is **not guaranteed**.

🟦 2.10 HashMap

HashMap is a class of the Java Collection Framework that implements the **Map** interface. It is available in the **java.util** package. HashMap is used to store data in the form of **key-value pairs**, where each key is unique and is associated with a single value. It uses **hashing** to store and retrieve data efficiently, providing fast insertion, deletion, and searching operations.

HashMap is internally **backed by a hash table**. It does not maintain the insertion order or sort the entries. It allows **one null key** and **multiple null values**. HashMap is widely used in applications where fast retrieval of values using keys is required.

🟦 Features of HashMap

- Implements the **Map** interface.
- Stores data in the form of **key-value pairs**.
- Keys are unique, while values can be duplicated.
- Uses **hashing** to store and retrieve data.
- Stores entries based on the **hash code** of keys.
- Is internally backed by a **hash table**.
- Does not maintain the insertion order.
- Does not sort the entries.
- Does not guarantee any predictable iteration order.
- Allows one **null** key.
- Allows multiple **null** values.
- Provides fast insertion, deletion, and searching operations.

🟦 Common Methods of HashMap

Method	Description
put(K key, V value)	Inserts a key-value pair into the HashMap.
get(Object key)	Returns the value associated with the specified key.
remove(Object key)	Removes the mapping for the specified key.
containsKey(Object key)	Checks whether the specified key exists.
containsValue(Object value)	Checks whether the specified value exists.
keySet()	Returns a set containing all keys.
values()	Returns a collection containing all values.
entrySet()	Returns a set containing all key-value mappings.
size()	Returns the number of key-value pairs.
isEmpty()	Checks whether the HashMap is empty.
clear()	Removes all key-value pairs from the HashMap.

🟦 Advantages of HashMap

- Stores data in the form of key-value pairs.
- Provides fast insertion, deletion, and searching operations.
- Allows one **null** key and multiple **null** values.
- Suitable for fast data retrieval using keys.
- Provides constant-time performance for basic operations under normal conditions.
- Efficient for storing and retrieving key-value mappings.

🟦 Limitations of HashMap

- Does not maintain the insertion order.
- Does not sort the entries.
- Does not guarantee any predictable iteration order.
- Keys must be unique.
- Not synchronized, so it is not thread-safe.
- Cannot be traversed using index values.
- Iteration order may change after insertion or deletion.

🟦 Program

```
import java.util.*;


public class HashMapDemo
{
   public static void main(String args[])
   {
      HashMap<Integer, String> map = new HashMap<Integer, String>();


      map.put(101, "Amit");
      map.put(102, "Rahul");
      map.put(103, "Priya");


      System.out.println("HashMap : " + map);
      System.out.println("Student with Key 102 : " + map.get(102));
      System.out.println("Contains Key 101 : " + map.containsKey(101));
      System.out.println("Contains Value Priya : " + map.containsValue("Priya"));
```

```
        System.out.println("Size : " + map.size());

        map.remove(102);
        System.out.println("After Removing Key 102 : " + map);

        System.out.println("Keys : " + map.keySet());
        System.out.println("Values : " + map.values());
        System.out.println("Entries : " + map.entrySet());
    }
}
```

Output

```
HashMap : {101=Amit, 102=Rahul, 103=Priya}
Student with Key 102 : Rahul
Contains Key 101 : true
Contains Value Priya : true
Size : 3
After Removing Key 102 : {101=Amit, 103=Priya}
Keys : [101, 103]
Values : [Amit, Priya]
Entries : [101=Amit, 103=Priya]
```

Note: The order of key-value pairs displayed in a **HashMap** may vary because HashMap does **not** maintain any predictable order of entries. Therefore, the output order is **not guaranteed**.

🟦 2.11 Hashtable

Hashtable is a class of the Java Collection Framework that implements the **Map** interface. It is available in the **java.util** package. Hashtable is used to store data in the form of **key-value pairs**, where each key is unique and is associated with a single value. It uses **hashing** to store and retrieve key-value pairs efficiently, providing fast insertion, deletion, and searching operations.

Hashtable is internally based on a **hash table**. It is **synchronized**, which makes it **thread-safe**. Unlike HashMap, Hashtable **does not allow null keys or null values**. It is a **legacy class** and is mainly used in multi-threaded applications where thread safety is required.

🟦 Features of Hashtable

- Implements the **Map** interface.
- Stores data in the form of **key-value pairs**.
- Keys are unique, while values can be duplicated.
- Uses **hashing** to store and retrieve data.
- Is internally based on a **hash table**.
- Is synchronized and thread-safe.
- Is a **legacy class** in Java.
- Does not maintain the insertion order.
- Does not sort the entries.
- Does not guarantee any predictable iteration order.
- Does not allow **null** keys or **null** values.
- Provides fast insertion, deletion, and searching operations.

🟦 Common Methods of Hashtable

Method	Description
put(K key, V value)	Inserts a key-value pair into the Hashtable.
get(Object key)	Returns the value associated with the specified key.
remove(Object key)	Removes the mapping for the specified key.
containsKey(Object key)	Checks whether the specified key exists.
containsValue(Object value)	Checks whether the specified value exists.
keys()	Returns an **Enumeration** of all keys in the Hashtable.
keySet()	Returns a set containing all keys.
values()	Returns a collection containing all values.
entrySet()	Returns a set containing all key-value mappings.
size()	Returns the number of key-value pairs.
isEmpty()	Checks whether the Hashtable is empty.
clear()	Removes all key-value pairs from the Hashtable.

🟦 Advantages of Hashtable

- Stores data in the form of key-value pairs.
- Is synchronized and thread-safe.
- Provides fast insertion, deletion, and searching operations.
- Suitable for multi-threaded applications.
- Suitable for applications requiring thread-safe key-value storage.
- Provides constant-time performance for basic operations under normal conditions.

🟦 Limitations of Hashtable

- Does not maintain the insertion order.
- Does not sort the entries.
- Does not guarantee any predictable iteration order.
- Does not allow **null** keys or **null** values.
- Slower than **HashMap** because of synchronization.
- It is a **legacy class**, and **HashMap** is preferred for single-threaded applications.

🟦 Program

```
import java.util.*;


public class HashtableDemo
{
   public static void main(String args[])
   {
      Hashtable<Integer, String> table = new Hashtable<Integer, String>();


      table.put(101, "Amit");
      table.put(102, "Rahul");
      table.put(103, "Priya");


      System.out.println("Hashtable : " + table);
      System.out.println("Student with Key 102 : " + table.get(102));
      System.out.println("Contains Key 101 : " + table.containsKey(101));
```

```
        System.out.println("Contains Value Priya : " +
table.containsValue("Priya"));
        System.out.println("Size : " + table.size());

        table.remove(102);
        System.out.println("After Removing Key 102 : " + table);

        System.out.println("Keys : " + table.keySet());
        System.out.println("Values : " + table.values());
        System.out.println("Entries : " + table.entrySet());

        Enumeration<Integer> e = table.keys();

        System.out.println("Keys using Enumeration :");

        while(e.hasMoreElements())
        {
           System.out.println(e.nextElement());
        }
    }
}
```

Output

```
Hashtable : {101=Amit, 102=Rahul, 103=Priya}
Student with Key 102 : Rahul
Contains Key 101 : true
Contains Value Priya : true
```

Size : 3

After Removing Key 102 : {101=Amit, 103=Priya}

Keys : [101, 103]

Values : [Amit, Priya]

Entries : [101=Amit, 103=Priya]

Keys using Enumeration :

101

103

Note: The order of key-value pairs displayed in a **Hashtable** may vary because Hashtable does **not** maintain any predictable iteration order. Therefore, the output order is **not guaranteed**.

🟦 2.12 TreeMap

TreeMap is a class of the Java Collection Framework that implements the **NavigableMap** interface, which extends the **SortedMap** interface and indirectly implements the **Map** interface. It is available in the **java.util** package. TreeMap is used to store data in the form of **key-value pairs**, where each key is unique and is automatically stored in **natural (ascending) order**.

TreeMap is internally **implemented using a Red-Black Tree**. It provides efficient searching, insertion, and deletion operations. It does not maintain the insertion order, does not allow duplicate keys, and does not allow **null** keys. However, it allows **multiple null values**. TreeMap is suitable for applications where data needs to be stored in sorted order.

Features of TreeMap

- Implements the **NavigableMap** interface.
- Stores data in the form of **key-value pairs**.
- Keys are unique, while values can be duplicated.
- Automatically stores keys in **natural (ascending) order**.
- Does not allow duplicate keys.
- Uses the **Red-Black Tree** data structure.
- Does not maintain the insertion order.
- Does not allow **null** keys.
- Allows multiple **null** values.
- Is not synchronized and is not thread-safe.
- Provides efficient searching, insertion, and deletion operations.
- Supports navigation methods for accessing sorted data.

Common Methods of TreeMap

Method	Description
put(K key, V value)	Inserts a key-value pair into the TreeMap.
get(Object key)	Returns the value associated with the specified key.
remove(Object key)	Removes the mapping for the specified key.
containsKey(Object key)	Checks whether the specified key exists.
containsValue(Object value)	Checks whether the specified value exists.
firstKey()	Returns the first (lowest) key.
lastKey()	Returns the last (highest) key.
higherKey(K key)	Returns the least key greater than the specified key.

lowerKey(K key)	Returns the greatest key less than the specified key.
entrySet()	Returns a set containing all key-value mappings.
size()	Returns the number of key-value pairs.
isEmpty()	Checks whether the TreeMap is empty.
clear()	Removes all key-value pairs from the TreeMap.

🟦 Advantages of TreeMap

- Automatically stores keys in sorted order.
- Provides efficient searching, insertion, and deletion operations.
- Supports navigation methods such as **firstKey()**, **lastKey()**, **higherKey()**, and **lowerKey()**.
- Suitable for applications requiring sorted and unique keys.
- Allows multiple **null** values.
- Provides efficient retrieval of data based on sorted keys.

🟦 Limitations of TreeMap

- Does not maintain the insertion order.
- Does not allow **null** keys.
- Does not allow duplicate keys.
- Slower than **HashMap** because keys are maintained in sorted order.
- Requires keys to be mutually comparable.
- Is not synchronized and is not thread-safe.
- Does not support index-based access.

Program

```
import java.util.*;

public class TreeMapDemo
{
   public static void main(String args[])
   {
      TreeMap<Integer, String> map = new TreeMap<Integer, String>();

      map.put(103, "Priya");
      map.put(101, "Amit");
      map.put(102, "Rahul");

      System.out.println("TreeMap : " + map);
      System.out.println("First Key : " + map.firstKey());
      System.out.println("Last Key : " + map.lastKey());
      System.out.println("Higher Key (101) : " + map.higherKey(101));
      System.out.println("Lower Key (103) : " + map.lowerKey(103));
      System.out.println("Contains Key 102 : " + map.containsKey(102));
      System.out.println("Size : " + map.size());

      map.remove(102);
      System.out.println("After Removing Key 102 : " + map);

      System.out.println("Entries : " + map.entrySet());
   }
```

}

Output

TreeMap : {101=Amit, 102=Rahul, 103=Priya}

First Key : 101

Last Key : 103

Higher Key (101) : 102

Lower Key (103) : 102

Contains Key 102 : true

Size : 3

After Removing Key 102 : {101=Amit, 103=Priya}

Entries : [101=Amit, 103=Priya]

Note: TreeMap automatically stores keys in **natural (ascending) order**. Therefore, the output is displayed in **sorted key order**, not in the order of insertion.

🟦 Unit III: Java Database Connectivity (JDBC)

🟦 3.1 Introduction to JDBC

JDBC (Java Database Connectivity) is a standard Java API used to connect Java applications with relational databases. It provides a set of interfaces and classes that enable Java programs to establish a connection with a database, execute SQL queries, and retrieve or update data.

JDBC is a part of the **java.sql** package and acts as a bridge between a Java application and a Database Management System (DBMS). It supports various relational databases such as **MySQL, Oracle, PostgreSQL, SQL Server**, and others through JDBC drivers.

Using JDBC, a Java application can perform database operations such as **creating tables, inserting records, updating records, deleting records, and retrieving data**. JDBC provides a common programming interface that makes it easier to work with different relational databases by using the appropriate JDBC driver.

The basic steps involved in JDBC are:

- Import the **java.sql** package.
- Load the JDBC Driver.
- Establish a database connection.
- Create a **Statement** or **PreparedStatement** object.
- Execute SQL queries.
- Process the **ResultSet** (if required).
- Close the database connection.

Program

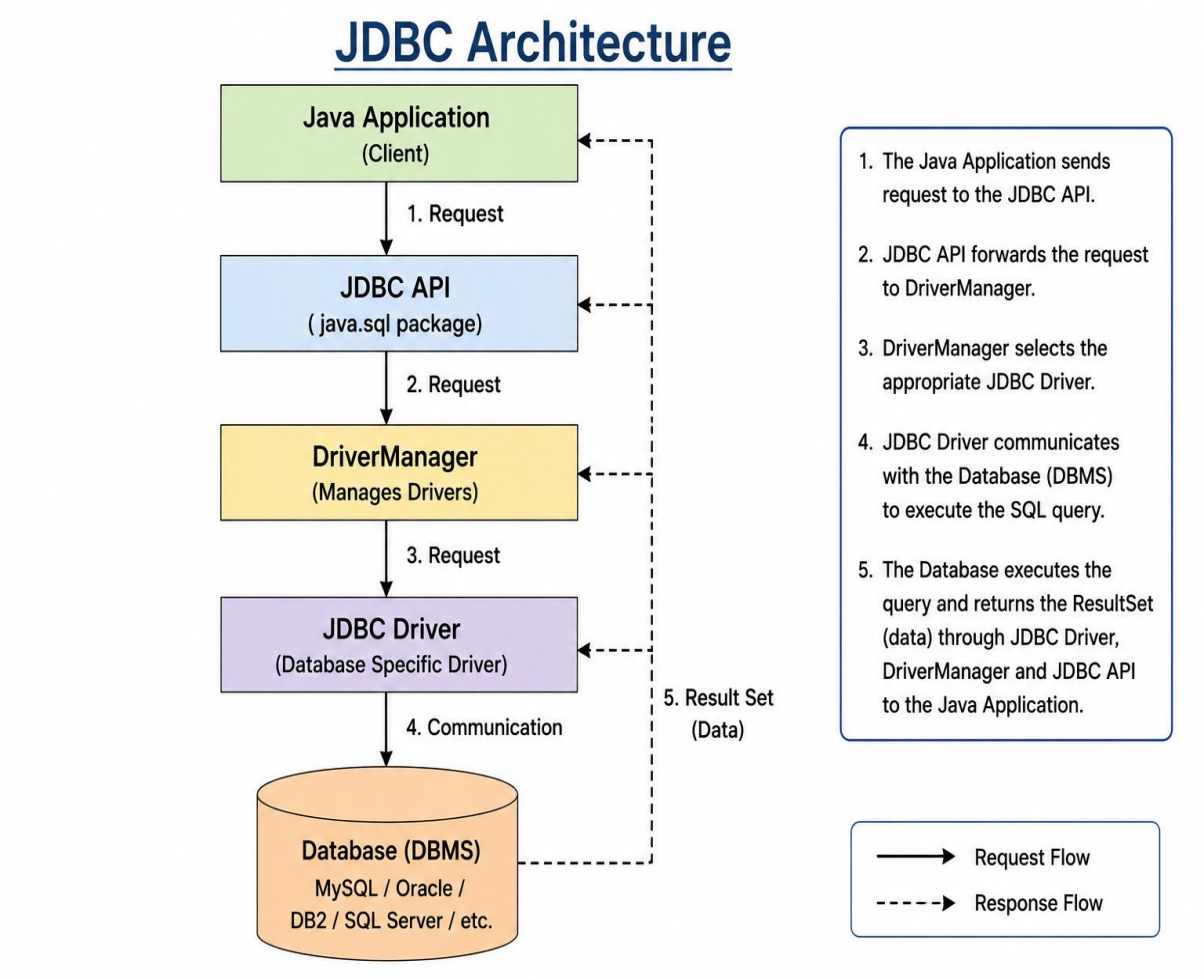
```
import java.sql.*;

public class JDBCDemo
{
   public static void main(String args[])
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");

         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb",
            "root",
            "password");

         if(con != null)
         {
            System.out.println("Database Connected Successfully.");
         }

         con.close();
      }
      catch(Exception e)
      {
         System.out.println(e);
```

```
        }
    }
}
```

Output

Database Connected Successfully.

🟦 3.2 JDBC Architecture



JDBC Architecture defines the interaction between a Java application and a database. It provides a standard mechanism for connecting Java programs to different relational databases using JDBC drivers. The architecture acts as a bridge between the Java application and the Database Management System (DBMS), enabling database connectivity and communication.

The JDBC Architecture consists of the following components:

- **Java Application:** The client application that sends database requests and processes the results.
- **JDBC API:** A set of interfaces and classes provided by the **java.sql** package that enables Java applications to communicate with databases.
- **DriverManager:** Manages JDBC drivers and establishes a connection between the Java application and the database.
- **JDBC Driver:** A software component that converts JDBC calls into database-specific commands and communicates with the database.
- **Database (DBMS):** The database system such as **MySQL, Oracle, PostgreSQL,** or **SQL Server**, where data is stored and managed.

Working of JDBC Architecture

1. The Java application sends a request to access the database.
2. The JDBC API receives the request and forwards it to the **DriverManager**.
3. The DriverManager selects the appropriate JDBC Driver.
4. The JDBC Driver establishes communication with the database.
5. The database executes the SQL query and returns the result.
6. The JDBC Driver sends the result to the JDBC API.
7. The JDBC API returns the result to the Java application in the form of a **ResultSet**.

Note: JDBC Architecture is based on the communication between the **Java Application**, **JDBC API**, **DriverManager**, **JDBC Driver**, and the **Database**, which together enable database connectivity and data processing in Java applications.

3.3 JDBC Drivers

A **JDBC Driver** is a software component that enables a Java application to communicate with a relational database. It converts JDBC API calls into database-specific commands and sends them to the database for execution. Different databases require different JDBC drivers. Java provides **four types of JDBC drivers**, each with a different mechanism for connecting to a database.

Type 1 Driver (JDBC-ODBC Bridge Driver)

The **Type 1 Driver** converts JDBC calls into **ODBC** calls, which are then processed by the ODBC driver to communicate with the database. It requires the ODBC driver to be installed on the client machine. This driver is platform-dependent, provides low performance, and has been removed from **Java 8 and later versions**.

Type 2 Driver (Native-API Driver)

The **Type 2 Driver** converts JDBC calls into database-specific native API calls. It requires native database libraries to be installed on the client machine. It provides better performance than the Type 1 Driver but is platform-dependent because it depends on native libraries.

Type 3 Driver (Network Protocol Driver)

The **Type 3 Driver** sends JDBC requests to a middleware server, which converts them into database-specific protocols before communicating with the database. It does not require native libraries on the client side and can connect to multiple databases through a single middleware server. However, it requires an additional middleware layer.

🟦 Type 4 Driver (Thin Driver)

The **Type 4 Driver** is a pure Java driver that directly communicates with the database using its native protocol. It does not require ODBC, native libraries, or middleware. It is platform-independent, provides the best performance, and is the most widely used JDBC driver in modern Java applications.

🟦 3.4 Establishing Database Connection

Establishing a Database Connection is the process of creating a communication link between a Java application and a database using JDBC. Before performing any database operation, a connection must be established. The **DriverManager** class is used to establish a connection between the Java application and the database.

The **getConnection()** method of the **DriverManager** class is used to create a database connection. It requires the database URL, username, and password. If the connection is established successfully, it returns a **Connection** object, which is used to execute SQL queries.

Steps to Establish a Database Connection

1. Import the **java.sql** package.
2. Load the JDBC Driver.
3. Establish the database connection using **DriverManager.getConnection()**.
4. Obtain the **Connection** object.
5. Close the connection after completing the database operations.

Syntax

```
Connection con = DriverManager.getConnection(
   "jdbc:mysql://localhost:3306/studentdb",
   "root",
   "password");
```

Program

```
import java.sql.*;


public class DatabaseConnectionDemo
{
   public static void main(String args[])
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb",
            "root",
            "password");


         System.out.println("Database Connected Successfully.");
         con.close();
      }
      catch(Exception e)
      {
         System.out.println(e);
      }
   }
}
```

Output

```
Database Connected Successfully.
```

🟦 3.5 Executing SQL Query

Executing an SQL Query is the process of sending SQL statements from a Java application to the database using JDBC. After establishing a database connection, SQL statements are executed using the **Statement** or **PreparedStatement** interface. These statements are used to **insert, update, delete,** and **retrieve** data from the database.

JDBC provides different methods to execute SQL statements depending on the type of SQL query. A **SELECT** statement is used to retrieve data from the database, while **INSERT, UPDATE, DELETE**, and **DDL statements** such as **CREATE, ALTER,** and **DROP** are used to modify the database structure or data. The execution result may be returned as a **ResultSet** object or as the number of rows affected by the SQL statement.

Methods for Executing SQL Queries	
Method	**Description**
executeQuery()	Executes a **SELECT** statement and returns the result as a **ResultSet** object.
executeUpdate()	Executes **INSERT, UPDATE, DELETE**, and **DDL statements** such as **CREATE, ALTER,** and **DROP**. It returns the number of affected rows.
execute()	Executes any SQL statement and returns **true** if the result is a **ResultSet**, otherwise **false**.

Syntax

```
Statement stmt = con.createStatement();


ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

Program

```
import java.sql.*;

public class ExecuteQueryDemo
{
   public static void main(String args[])
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");

         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb",
            "root",
            "password");

         Statement stmt = con.createStatement();

         ResultSet rs = stmt.executeQuery("SELECT * FROM student");

         while(rs.next())
         {
            System.out.println(rs.getInt("id") + " " +
                        rs.getString("name"));
         }
```

```
            rs.close();
            stmt.close();
            con.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output

101 Amit

102 Rahul

103 Priya

Note: executeQuery() is used only for **SELECT** statements, whereas executeUpdate() is used for **INSERT, UPDATE, DELETE**, and **DDL statements** such as **CREATE, ALTER,** and **DROP**. The execute() method can execute any type of SQL statement and is mainly used when the type of SQL statement is not known in advance.

🟦 3.6 Processing Results (ResultSet)

ResultSet is an interface in the **java.sql** package that is used to store and process the data returned by a **SELECT** SQL query. When the executeQuery() method is executed, it returns a **ResultSet** object containing the records retrieved from the database.

A **ResultSet** object maintains a **cursor**, which initially points **before the first row** of the result. The next() method moves the cursor to the next row, allowing the application to read one record at a time. Data from each column can be retrieved using methods such as getInt(), getString(), getFloat(), and getDouble().

A ResultSet object is mainly used for reading database records and displaying them in a Java application. It supports cursor navigation and provides methods to retrieve column values according to their respective data types.

🟦 Features of ResultSet

- Stores the records returned by a **SELECT** query.
- Returned by the executeQuery() method.
- Maintains a cursor to navigate through records.
- Allows reading one row at a time.
- Retrieves data using column names or column indexes.
- Supports different data types such as **int, String, float,** and **double**.
- Provides methods for cursor navigation and data retrieval.

Common Methods of ResultSet	
Method	**Description**
next()	Moves the cursor to the next row.
previous()	Moves the cursor to the previous row.
first()	Moves the cursor to the first row.
last()	Moves the cursor to the last row.
getInt(String columnName)	Returns the integer value of the specified column.
getString(String columnName)	Returns the string value of the specified column.
getFloat(String columnName)	Returns the float value of the specified column.
getDouble(String columnName)	Returns the double value of the specified column.
close()	Closes the ResultSet object.

Program

```
import java.sql.*;


public class ResultSetDemo
{
   public static void main(String args[])
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            Connection con = DriverManager.getConnection(
               "jdbc:mysql://localhost:3306/studentdb",
               "root",
               "password");

            Statement stmt = con.createStatement();

            ResultSet rs = stmt.executeQuery("SELECT * FROM student");

            while(rs.next())
            {
               System.out.println(rs.getInt("id") + " " +
                                  rs.getString("name"));
            }

            rs.close();
            stmt.close();
            con.close();
         }
         catch(Exception e)
         {
            System.out.println(e);
         }
      }
   }
```

Output

101 Amit

102 Rahul

103 Priya

Note: A **ResultSet** object is used only with the executeQuery() method to process the records returned by a **SELECT** statement.

🟦 3.7 Metadata

Metadata is the information that describes the structure and properties of a database or the data stored in it. In JDBC, metadata provides details about the database, tables, columns, data types, and query results. It helps Java applications obtain information about the database without directly accessing the actual data.

JDBC provides two types of metadata: **DatabaseMetaData** and **ResultSetMetaData**. These interfaces provide information about the database and the query result, making it easier to develop database applications.

🟦 Types of Metadata

1. DatabaseMetaData

The **DatabaseMetaData** interface provides information about the database, JDBC driver, database version, tables, schemas, and supported features. It is obtained by using the getMetaData() method of the **Connection** interface.

2. ResultSetMetaData

The **ResultSetMetaData** interface provides information about the columns of a **ResultSet**, such as the column name, data type, column size, and number of columns. It is obtained by using the getMetaData() method of the **ResultSet** interface.

Common Methods of Metadata	
Method	**Description**
getDatabaseProductName()	Returns the name of the database.
getDriverName()	Returns the name of the JDBC driver.
getDatabaseProductVersion()	Returns the database version.
getColumnCount()	Returns the number of columns in the ResultSet.
getColumnName(int column)	Returns the name of the specified column.
getColumnType(int column)	Returns the SQL data type of the specified column.
getColumnTypeName(int column)	Returns the data type name of the specified column.

Program

```
import java.sql.*;


public class MetadataDemo
{
   public static void main(String args[])
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");


         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb",
```

```
            "root",
            "password");

        DatabaseMetaData dbmd = con.getMetaData();

        System.out.println("Database : " + dbmd.getDatabaseProductName());
        System.out.println("Driver : " + dbmd.getDriverName());
        System.out.println("Version : " + dbmd.getDatabaseProductVersion());

        Statement stmt = con.createStatement();

        ResultSet rs = stmt.executeQuery("SELECT * FROM student");

        ResultSetMetaData rsmd = rs.getMetaData();

        System.out.println("Number of Columns : " + rsmd.getColumnCount());
        System.out.println("First Column Name : " + rsmd.getColumnName(1));

        rs.close();
        stmt.close();
        con.close();
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
```

```
    }
}
```

Output

Database : MySQL

Driver : MySQL Connector/J

Version : 8.0.xx

Number of Columns : 2

First Column Name : id

Note: DatabaseMetaData provides information about the database and JDBC driver, whereas **ResultSetMetaData** provides information about the columns of a **ResultSet** object returned by a SQL query.

🟦 3.8 PreparedStatement

PreparedStatement is an interface in the **java.sql** package that is used to execute **precompiled and parameterized SQL statements** in JDBC. It is created using the prepareStatement() method of the **Connection** interface. Unlike the **Statement** interface, a PreparedStatement is compiled only once by the database and can be executed multiple times with different parameter values.

PreparedStatement uses **placeholders (?)** to represent input values. These values are assigned at runtime using setter methods such as setInt(), setString(), setFloat(), and setDouble(). Since user input is treated as data rather than executable SQL code, PreparedStatement helps prevent **SQL Injection** attacks and improves application security.

PreparedStatement is commonly used for executing **INSERT, UPDATE, DELETE,** and **SELECT** queries repeatedly with different values. It provides better performance, improves code readability, and is the preferred choice over the **Statement** interface in modern JDBC applications.

🟦 Working of PreparedStatement

1. Establish a database connection.
2. Create a PreparedStatement object using the prepareStatement() method.
3. Write the SQL query using **? (placeholders)** for input values.
4. Set the parameter values using appropriate setter methods.
5. Execute the SQL statement using executeQuery() or executeUpdate().
6. Process the result (if required).
7. Close the PreparedStatement and the database connection.

🟦 Syntax

```
String sql = "INSERT INTO student(id, name) VALUES(?, ?)";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1, 101);
ps.setString(2, "Amit");

ps.executeUpdate();
```

🟦 Advantages of PreparedStatement

- SQL statements are precompiled, improving execution performance.
- Prevents **SQL Injection** attacks.
- Supports parameterized SQL queries.
- Executes the same SQL statement multiple times with different values.
- Improves code readability and maintainability.
- Reduces the chances of SQL syntax errors.
- Widely used in enterprise Java applications.

🟦 Program

```
import java.sql.*;


public class PreparedStatementDemo
{
   public static void main(String args[])
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");


         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb",
            "root",
            "password");


         String sql = "INSERT INTO student(id, name) VALUES(?, ?)";
```

```
            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, 104);
            ps.setString(2, "Neha");

            int rows = ps.executeUpdate();

            System.out.println(rows + " Record Inserted Successfully.");

            ps.close();
            con.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output

1 Record Inserted Successfully.

Note: PreparedStatement is preferred over **Statement** because it is faster, more secure, supports parameterized queries, and helps prevent **SQL Injection** attacks.

🟦 3.9 CallableStatement

CallableStatement is an interface in the **java.sql** package that is used to execute **stored procedures** in a relational database. It is created using the prepareCall() method of the **Connection** interface. A **stored procedure** is a precompiled collection of SQL statements stored in the database that can be executed whenever required.

Unlike **Statement** and **PreparedStatement**, CallableStatement is specifically designed to execute stored procedures. It supports **IN**, **OUT**, and **INOUT** parameters, making it suitable for performing complex database operations and exchanging data between the Java application and the database.

CallableStatement is widely used in enterprise applications because stored procedures improve performance, reduce network traffic, enhance security, and allow business logic to be maintained within the database.

🟦 Parameters of CallableStatement

1. IN Parameter

An **IN** parameter is used to pass values from the Java application to the stored procedure. These values are assigned using setter methods such as setInt(), setString(), etc.

2. OUT Parameter

An **OUT** parameter is used to return values from the stored procedure to the Java application. It is registered using the registerOutParameter() method and retrieved using getter methods.

3. INOUT Parameter

An **INOUT** parameter is used both to send values to the stored procedure and receive modified values back from it.

🟦 Syntax

```
CallableStatement cs =
con.prepareCall("{call InsertStudent(?, ?)}");

cs.setInt(1, 101);
cs.setString(2, "Amit");

cs.execute();
```

🟦 Program

```
import java.sql.*;

public class CallableStatementDemo
{
   public static void main(String args[])
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");

         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb",
            "root",
            "password");
```

```
            CallableStatement cs =
                con.prepareCall("{call InsertStudent(?, ?)}");

            cs.setInt(1, 105);
            cs.setString(2, "Riya");

            cs.execute();

            System.out.println("Stored Procedure Executed Successfully.");

            cs.close();
            con.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output

Stored Procedure Executed Successfully.

Note: CallableStatement is used only for executing **stored procedures**. It supports **IN**, **OUT**, and **INOUT** parameters and is preferred when database operations are implemented as stored procedures.

🟦 Unit IV: Introduction to Servlets

🟦 4.1 Introduction to Servlets

A **Servlet** is a server-side Java component that is used to create **dynamic web applications**. It is a Java class that runs inside a **Servlet Container** (also known as a Web Container) such as **Apache Tomcat**. Servlets receive requests from clients, process those requests, and generate appropriate responses dynamically.

Servlets are a part of the **Java Servlet API** provided by Java Enterprise technology. They follow the **request-response model**, where a client (web browser) sends an HTTP request to the web server, the servlet processes the request, performs the required business logic, interacts with databases or other server-side resources if necessary, and returns an HTTP response to the client.

Unlike static HTML pages, servlets generate web pages dynamically according to user input or database information. Since a servlet is loaded into memory only once and handles multiple client requests using separate threads, it provides better performance and resource utilization than traditional **CGI (Common Gateway Interface)** programs.

Servlets are widely used for developing enterprise web applications such as **online banking systems, e-commerce websites, student management systems, content management systems, login authentication, form processing, session management,** and **database-driven web applications**.

🟦 Features of Servlets

- Servlets are written in Java and are platform-independent.
- They execute on the server side.
- They generate dynamic web content.
- They follow the HTTP request-response model.
- They handle multiple client requests efficiently using multithreading.
- They can interact with databases through JDBC.
- They support session management using Cookies and HttpSession.
- They provide better performance than CGI programs.
- They are secure, portable, scalable, and reusable.

Note: A **Servlet** acts as an intermediary between the **client (web browser)** and the **web server**, processing client requests and generating dynamic responses. It forms the foundation of Java web application development.

🟦 4.2 Deploying a Simple Servlet

Servlet Deployment is the process of configuring and placing a servlet in a **Servlet Container (Web Container)** so that it can receive client requests and generate responses. Before a servlet can be executed, it must be compiled, mapped with a URL, and deployed on a web server such as **Apache Tomcat**. Examples of servlet containers include **Apache Tomcat, GlassFish,** and **JBoss**.

In a Dynamic Web Project, the servlet class is created by extending the **HttpServlet** class. The servlet is then mapped to a URL either by using the **@WebServlet** annotation or the **web.xml** deployment descriptor. When a client accesses the specified URL through a web browser, the servlet container loads the servlet, executes it, and returns the generated response to the client.

A servlet can be deployed in two ways:

- **Using @WebServlet Annotation** (Recommended)
- **Using web.xml Deployment Descriptor**

Modern Java web applications generally use the **@WebServlet** annotation because it simplifies servlet configuration, whereas older applications commonly use the **web.xml** file.

🟦 Steps to Deploy a Simple Servlet

1. Install and configure **Apache Tomcat 9**.
2. Create a **Dynamic Web Project** in Eclipse.
3. Add the **Servlet API** library to the project.
4. Create a servlet class by extending the **HttpServlet** class.
5. Override the doGet() or doPost() method.
6. Configure URL mapping using **@WebServlet** or **web.xml**.
7. Deploy the project on the Tomcat server.
8. Start the Tomcat server.
9. Access the servlet using its URL in a web browser.

🟦 Simple Servlet Program

```
import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

import javax.servlet.annotation.*;
```

```
@WebServlet("/HelloServlet")
public class HelloServlet extends HttpServlet
{
   public void doGet(HttpServletRequest request,
                     HttpServletResponse response)
         throws ServletException, IOException
   {
      response.setContentType("text/html");

      PrintWriter out = response.getWriter();

      out.println("<h2>Hello, Welcome to Servlet</h2>");
   }
}
```

URL

http://localhost:8080/ProjectName/HelloServlet

Output

Hello, Welcome to Servlet

Note: A servlet can be deployed using either the **@WebServlet** annotation or the **web.xml** deployment descriptor. In **Apache Tomcat 9**, the **@WebServlet** annotation is commonly used because it simplifies servlet configuration and reduces the need for XML configuration.

🟦 4.3 Servlet Life Cycle

The **Servlet Life Cycle** describes the sequence of stages through which a servlet passes from its creation until it is removed from memory. The life cycle of a servlet is managed by the **Servlet Container (Web Container)** such as **Apache Tomcat**. The container is responsible for loading the servlet, creating its object, initializing it, processing client requests, and finally destroying it.

A servlet is loaded into memory only once. After initialization, the same servlet object handles multiple client requests using separate threads, making servlets efficient, scalable, and suitable for developing enterprise web applications.

The Servlet Life Cycle consists of the following three methods:

🟦 1. init() Method

The init() method is called **only once** when the servlet is loaded into memory for the first time. It is used to perform initialization tasks such as creating database connections, reading configuration parameters, or allocating resources. This method is executed before the servlet starts processing client requests.

Syntax

```
public void init() throws ServletException
{
   // Initialization code
}
```

🟦 2. service() Method

The service() method is called **every time** a client sends a request to the servlet. It receives the client's request, processes it, and generates the appropriate response. In the **HttpServlet** class, the service() method automatically invokes methods such as doGet(), doPost(), doPut(), and doDelete() depending on the type of HTTP request.

Syntax

```
public void service(ServletRequest request,
                    ServletResponse response)
      throws ServletException, IOException
{
   // Request processing code
}
```

🟦 3. destroy() Method

The destroy() method is called **only once** before the servlet is removed from memory by the servlet container. It is used to release resources such as database connections, files, or network resources before the servlet is destroyed.

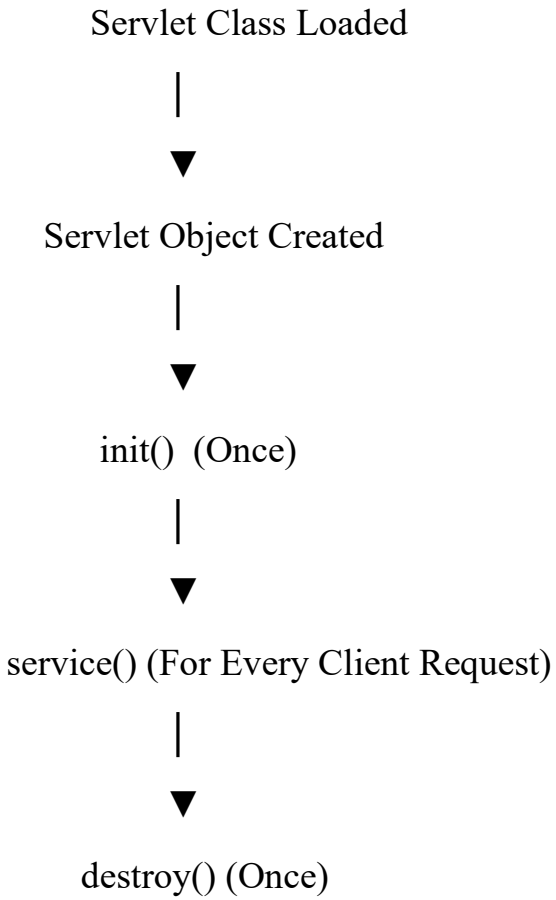
Syntax

```
public void destroy()
{
   // Cleanup code
}
```

🟦 Life Cycle Sequence

1. The servlet container loads the servlet class.
2. The servlet container creates an instance (object) of the servlet.
3. The init() method is called once.
4. The service() method is called for every client request.
5. The destroy() method is called once before the servlet is unloaded.

🟦 Servlet Life Cycle Diagram



Note: The **init()** and **destroy()** methods are executed only **once** during the servlet life cycle, whereas the **service()** method is executed for **every client request**. This allows a single servlet instance to efficiently handle multiple client requests.

🟦 4.4 GET Request

A **GET Request** is an HTTP request method used by a client (web browser) to **retrieve information** from the web server. It is the **default request method** used by web browsers to request resources from the server. In Servlets, a GET request is handled by overriding the **doGet()** method of the **HttpServlet** class.

When a GET request is sent, the request parameters are appended to the URL as a **query string** in the form of **name=value** pairs. The data is transmitted after the **? (question mark)** symbol in the URL. Since the data is visible in the browser's address bar, the GET method is suitable for sending **non-sensitive information** such as search keywords, page numbers, or filter values.

GET requests can be **bookmarked**, stored in the browser history, and cached by the browser. However, because the data is included in the URL, it should not be used for transmitting confidential information such as passwords, banking details, or personal information.

🟦 Characteristics of GET Request

- Used to retrieve data from the server.
- Handled by the doGet() method.
- Request data is appended to the URL.
- Parameters are sent as **query strings**.
- Data is transmitted after the **? (question mark)** symbol in the URL.
- Data is visible in the browser's address bar.
- Can be bookmarked and cached.
- Suitable for sending non-sensitive information.
- Has a limited data length depending on the browser and server.

🟦 Syntax

```
public void doGet(HttpServletRequest request,
                  HttpServletResponse response)
      throws ServletException, IOException
{
   // Process GET request
}
```

Program

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/GetDemo")
public class GetDemo extends HttpServlet
{
   public void doGet(HttpServletRequest request,
                     HttpServletResponse response)
          throws ServletException, IOException
   {
      response.setContentType("text/html");

      PrintWriter out = response.getWriter();

      String name = request.getParameter("name");

      out.println("<h2>Welcome " + name + "</h2>");
   }
}
```

URL

http://localhost:8080/ProjectName/GetDemo?name=Amit

Output

Welcome Amit

Note: A **GET Request** is mainly used to retrieve data from the server. Since request parameters are visible in the URL, it should **not** be used for sending sensitive information such as passwords or personal data.

🟦 4.5 POST Request

A **POST Request** is an HTTP request method used to **send data from the client to the server** for processing. In Servlets, a POST request is handled by overriding the **doPost()** method of the **HttpServlet** class.

Unlike the GET method, the data in a POST request is sent in the **body of the HTTP request** rather than in the URL. Therefore, the submitted data is **not visible** in the browser's address bar. This makes the POST method more suitable for transmitting **sensitive information** such as passwords, login credentials, registration details, personal information, and financial data.

The POST method is commonly used when the client needs to **submit data to the server for processing or storage**, such as user registration, login authentication, feedback forms, and database insertion. It does not have the URL length limitation associated with the GET method and can transmit a large amount of data efficiently.

🟦 Characteristics of POST Request

- Used to send data from the client to the server.
- Handled by the doPost() method.
- Request data is sent in the body of the HTTP request.
- Data is not visible in the browser's address bar.
- Suitable for sending sensitive and confidential information.
- Can send a large amount of data.
- Cannot normally be bookmarked or cached.
- Commonly used for registration forms, login forms, feedback forms, and online transactions.

🟦 HTML Form

```
<form action="PostDemo" method="post">
   Name:
   <input type="text" name="name">
   <input type="submit" value="Submit">
</form>
```

🟦 Syntax

```
public void doPost(HttpServletRequest request,
                   HttpServletResponse response)
      throws ServletException, IOException
{
   // Process POST request
}
```

🟦 Servlet Program

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/PostDemo")
public class PostDemo extends HttpServlet
{
   public void doPost(HttpServletRequet request,
                      HttpServletResponse response)
```

```
        throws ServletException, IOException
    {
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");

        out.println("<h2>Welcome " + name + "</h2>");
    }
}
```

Output

Welcome Amit

Difference Between GET and POST Request

GET Request	POST Request
Used to retrieve data from the server.	Used to send data to the server.
Handled by the doGet() method.	Handled by the doPost() method.
Parameters are sent through the URL.	Parameters are sent in the request body.
Data is visible in the address bar.	Data is not visible in the address bar.
Suitable for non-sensitive data.	Suitable for sensitive data.
Has limited data length.	Can send a large amount of data.
Can be bookmarked and cached.	Cannot normally be bookmarked or cached.

Note: The **POST** method is preferred for form submission and transmitting confidential information because the request data is sent in the **HTTP request body** instead of the URL.

🟦 4.6 Request Object

The **Request Object** is an object of the **HttpServletRequest** interface that represents the client's HTTP request sent to a servlet. It is automatically created by the **Servlet Container** and passed as a parameter to the **doGet()** or **doPost()** method whenever a client sends a request.

The Request Object contains all the information associated with the client's request, such as **request parameters, form data, HTTP headers, cookies, session information, request attributes,** and **client details**. It enables a servlet to read and process the data submitted by the client.

The Request Object plays an important role in web applications because it allows the servlet to retrieve user input, identify the client, access request-related information, and pass data to other server-side resources.

🟦 Common Methods of HttpServletRequest

Method	Description
getParameter(String name)	Returns the value of the specified request parameter.
getParameterNames()	Returns the names of all request parameters.
getParameterValues(String name)	Returns multiple values of the specified parameter.
getAttribute(String name)	Returns the value of the specified request attribute.
setAttribute(String name, Object value)	Stores an object as a request attribute.

getHeader(String name)	Returns the value of the specified HTTP header.
getMethod()	Returns the HTTP request method (GET or POST).
getRequestURI()	Returns the URI of the requested resource.
getRemoteAddr()	Returns the IP address of the client.

🟦 HTML Form

```
<form action="RequestDemo" method="post">
    Name:
    <input type="text" name="name">
    <input type="submit" value="Submit">
</form>
```

🟦 Servlet Program

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/RequestDemo")
public class RequestDemo extends HttpServlet
{
    public void doPost(HttpServletRequest request,
                       HttpServletResponse response)
            throws ServletException, IOException
```

```
    {
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");

        out.println("<h2>Welcome " + name + "</h2>");
    }
}
```

Output

Welcome Amit

Note: The **HttpServletRequest** object is used to obtain information sent by the client, including form data, request parameters, HTTP headers, cookies, and other request-related details. It is automatically created by the **Servlet Container** and passed to the servlet whenever an HTTP request is received.

Unit V: Handling Form Data

5.1 Accessing Data from HTML Form

HTML forms provide an easy and interactive way for users to enter information and submit it to a web application. In Servlet-based applications, an HTML form serves as the interface through which users enter data such as **name, email, password, age, feedback,** or **search keywords**. When the user clicks the **Submit** button, the browser sends the entered data to the servlet specified in the **action** attribute of the <form> tag.

The **method** attribute determines how the data is transmitted to the server. The **GET** method appends the data to the URL, whereas the **POST** method sends the data in the body of the HTTP request. After receiving the request, the servlet accesses the submitted values through the **HttpServletRequest** object.

Each form element must have a **name** attribute because the servlet identifies and retrieves the submitted data using this name. The getParameter() method is commonly used to retrieve a single value, while getParameterValues() retrieves multiple values, and getParameterNames() returns the names of all submitted parameters.

Accessing data from HTML forms is one of the fundamental operations in Java web applications. It is widely used in **user registration, login authentication, online examination systems, e-commerce websites, search applications, feedback forms,** and **database-driven applications**.

🟦 Process of Accessing Data from HTML Form

1. The user enters data into the HTML form.
2. The browser sends the form data to the servlet using the **GET** or **POST** method.
3. The servlet receives the request through the **HttpServletRequest** object.
4. The submitted data is accessed using methods such as getParameter().
5. The servlet processes the received data.
6. The servlet generates and sends the response back to the client.

🟦 HTML Form Example

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Registration</title>
</head>
<body>

<form action="FormDataServlet" method="post">

   Name :
   <input type="text" name="name"><br><br>

   Email :
   <input type="email" name="email"><br><br>

   <input type="submit" value="Submit">
```

```
</form>

</body>
</html>
```

🟦 Servlet Example

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/FormDataServlet")
public class FormDataServlet extends HttpServlet
{
   public void doPost(HttpServletRequest request,
                      HttpServletResponse response)
         throws ServletException, IOException
   {
      response.setContentType("text/html");

      PrintWriter out = response.getWriter();

      String name = request.getParameter("name");
      String email = request.getParameter("email");
```

```
        out.println("<h2>Student Details</h2>");
        out.println("Name : " + name + "<br>");
        out.println("Email : " + email);
    }
}
```

Output

Student Details

Name : Amit

Email : amit@gmail.com

Note: While accessing HTML form data, the **name** attribute of each form field is mandatory because the servlet retrieves the submitted values using the methods of the **HttpServletRequest** interface based on these parameter names.

🟦 5.2 Using JDBC in Servlet

JDBC (Java Database Connectivity) is a Java API used to connect Java applications with relational databases such as **MySQL, Oracle, PostgreSQL,** and **SQL Server**. In Servlet-based web applications, JDBC enables a servlet to interact with the database for storing, retrieving, updating, and deleting data.

When a user submits information through an HTML form, the servlet receives the data using the **HttpServletRequest** object. The servlet then establishes a database connection using the **JDBC Driver** and **DriverManager**, executes the required SQL statement, processes the result, and generates an appropriate response for the client. This integration allows web applications to handle dynamic and database-driven operations efficiently.

Using JDBC in Servlets is widely applied in **user registration systems, login authentication, online banking, e-commerce websites, student management systems,** and **content management applications**, where data must be permanently stored and managed in a database.

🟦 Working of JDBC in Servlet

1. The user submits data through an HTML form.
2. The servlet receives the request using the **HttpServletRequest** object.
3. The servlet establishes a connection with the database using the **JDBC Driver** and **DriverManager**.
4. An SQL statement is executed using **PreparedStatement** or **Statement**.
5. The database processes the SQL statement.
6. The servlet processes the result returned by the database.
7. The servlet sends an appropriate response back to the client.

🟦 Servlet Program

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/RegisterServlet")
public class RegisterServlet extends HttpServlet
{
   public void doPost(HttpServletRequest request,
                      HttpServletResponse response)
         throws ServletException, IOException
   {
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
```

```
String name = request.getParameter("name");
String email = request.getParameter("email");

try
{
   Class.forName("com.mysql.cj.jdbc.Driver");

   Connection con = DriverManager.getConnection(
      "jdbc:mysql://localhost:3306/studentdb",
      "root",
      "password");

   PreparedStatement ps = con.prepareStatement(
      "INSERT INTO student(name, email) VALUES(?, ?)");

   ps.setString(1, name);
   ps.setString(2, email);

   int rows = ps.executeUpdate();

   if(rows > 0)
      out.println("Record Inserted Successfully.");
   else
      out.println("Record Not Inserted.");
```

```
            ps.close();
            con.close();
        }
        catch(Exception e)
        {
            out.println(e);
        }
    }
}
```

Output

Record Inserted Successfully.

Note: JDBC enables a servlet to communicate with a relational database. In Servlet-based applications, **PreparedStatement** is preferred over **Statement** because it improves performance, supports parameterized SQL queries, and helps prevent **SQL Injection** attacks.

🟦 5.3 Servlet Chaining

Servlet Chaining is a technique in which the output or request of one servlet is passed to another servlet for further processing. In other words, multiple servlets work together sequentially to complete a single client request. This approach improves modularity, code reusability, and maintainability by dividing a complex task into smaller and independent servlets.

Servlet chaining is commonly implemented using the **RequestDispatcher** interface. The forward() method transfers the request from one servlet to another without involving the client again, while the include() method includes the output of one servlet in the response generated by another servlet.

Servlet chaining is widely used in web applications such as **login authentication, online payment systems, shopping carts, report generation,** and **multi-step form processing**, where different servlets perform different tasks before generating the final response.

Working of Servlet Chaining

1. The client sends a request to the first servlet.
2. The first servlet processes the request.
3. The first servlet obtains a **RequestDispatcher** object.
4. The request is forwarded to another servlet using the forward() method or its output is included using the include() method.
5. The second servlet processes the request.
6. The final response is sent back to the client.

Methods Used in Servlet Chaining

Method	Description
forward(ServletRequest, ServletResponse)	Transfers the request and response to another servlet or resource.
include(ServletRequest, ServletResponse)	Includes the output of another servlet or resource in the current response.

Servlet Program

First Servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/FirstServlet")
public class FirstServlet extends HttpServlet
{
   public void doGet(HttpServletRequest request,
```

```
                HttpServletResponse response)
        throws ServletException, IOException
    {
        RequestDispatcher rd =
            request.getRequestDispatcher("SecondServlet");

        rd.forward(request, response);
    }
}
```

Second Servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/SecondServlet")
public class SecondServlet extends HttpServlet
{
   public void doGet(HttpServletRequest request,
                HttpServletResponse response)
        throws ServletException, IOException
    {
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
```

```
        out.println("<h2>Request Received by Second Servlet</h2>");
    }
}
```

Output

Request Received by Second Servlet

Note: Servlet Chaining allows multiple servlets to work together for processing a single client request. It is commonly implemented using the **RequestDispatcher** interface through the **forward()** and **include()** methods, resulting in better modularity and code reusability.

🟦 5.4 Cookies

A **Cookie** is a small piece of information stored on the client's web browser by a web server. Cookies are used to **maintain the state of a user** between multiple HTTP requests. Since HTTP is a **stateless protocol**, it does not remember information about previous requests. Cookies overcome this limitation by storing user-specific information on the client side.

Each cookie is stored as a **name-value pair** and is automatically sent by the browser to the server whenever the user revisits the same website. In Servlets, cookies are represented by the **Cookie** class available in the **javax.servlet.http** package. They are commonly used to store information such as **user preferences, login status, session identifiers, shopping cart details,** and **language settings**.

Cookies improve the user experience by remembering user information across multiple requests. However, since cookies are stored on the client machine, they should not be used to store highly sensitive information such as passwords or confidential financial data.

🟦 Types of Cookies

1. Session Cookie

A **Session Cookie** is stored temporarily in the browser and exists only for the duration of the user's browsing session. It is automatically deleted when the browser is closed.

2. Persistent Cookie

A **Persistent Cookie** is stored on the client machine for a specified period. Its lifetime is defined using the setMaxAge() method. It remains available even after the browser is closed until it expires or is deleted.

🟦 Working of Cookies

1. The client sends a request to the web server.
2. The servlet creates a **Cookie** object.
3. The servlet adds the cookie to the response using the addCookie() method.
4. The browser stores the cookie on the client machine.
5. During subsequent requests, the browser automatically sends the cookie back to the server.
6. The servlet retrieves the cookies using the getCookies() method and processes the stored information.

🟦 Common Methods of Cookie Class

Method	Description
Cookie(String name, String value)	Creates a new cookie.
setMaxAge(int expiry)	Sets the lifetime of the cookie in seconds.
getName()	Returns the cookie name.
getValue()	Returns the cookie value.
setValue(String value)	Updates the cookie value.
getCookies()	Returns all cookies sent by the client.

🟦 Servlet Program

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/CookieDemo")
public class CookieDemo extends HttpServlet
{
   public void doGet(HttpServletRequest request,
                     HttpServletResponse response)
         throws ServletException, IOException
   {
      response.setContentType("text/html");

      PrintWriter out = response.getWriter();

      Cookie cookie = new Cookie("username", "Amit");

      cookie.setMaxAge(3600);

      response.addCookie(cookie);

      out.println("Cookie Created Successfully.");
   }
}
```

Output

Cookie Created Successfully.

Note: Cookies are stored on the **client-side** and are mainly used to maintain user information across multiple HTTP requests. Since cookies can be viewed or modified by the client, they should **not** be used to store sensitive information such as passwords or confidential data.

🟦 5.5 Session Management

Session Management is the process of maintaining user-specific information across multiple HTTP requests in a web application. Since **HTTP is a stateless protocol**, it does not remember information about previous client requests. Session management enables the server to identify the same user throughout a browsing session and maintain the continuity of user interaction.

A **session** starts when a user first accesses a web application and ends when the user logs out, closes the browser, or the session expires due to inactivity. Session management is essential for preserving user information such as login details, shopping cart contents, user preferences, and authentication status across different web pages.

Without session management, every request is treated as a new request, making it impossible for web applications to recognize users or maintain their activities. Therefore, session management is an essential feature of modern web applications such as **online banking, e-commerce websites, online examination systems, student portals, hospital management systems,** and **social networking sites**.

🟦 Need of Session Management

- Maintains user information across multiple requests.
- Identifies the same user throughout a session.
- Preserves login and authentication details.
- Stores shopping cart information.
- Supports personalized user services.
- Improves security and user experience.
- Maintains application state during user interaction.

🟦 Session Tracking Techniques

Servlets provide **four commonly used techniques** for maintaining user sessions. The choice of a technique depends on the application requirements, performance, and security considerations.

1. Cookies

Cookies store user-related information on the **client-side**. The browser automatically sends the stored cookie with every request to the server.

2. URL Rewriting

URL Rewriting appends the **Session ID** to the URL. It is mainly used when cookies are disabled in the client's browser.

Example

http://localhost:8080/Project/LoginServlet;jsessionid=AB123XYZ

3. Hidden Form Fields

Hidden Form Fields use the HTML <input type="hidden"> element to store session-related information. These values are submitted along with the form data without being displayed to the user.

4. HttpSession

HttpSession is the most commonly used session tracking technique in Servlets. It stores session information on the **server-side** and assigns a unique **Session ID** to each user, providing better security and reliability.

🟦 Working of Session Management

1. The client sends the first request to the web server.
2. The servlet container creates a new session for the client.
3. A unique **Session ID** is generated.
4. The Session ID is sent to the client's browser.
5. The browser stores the Session ID and sends it with every subsequent request.
6. The server identifies the client using the Session ID.
7. The requested information is processed and the response is returned to the client.
8. The session ends when the user logs out or the session expires.

🟦 HttpSession Example

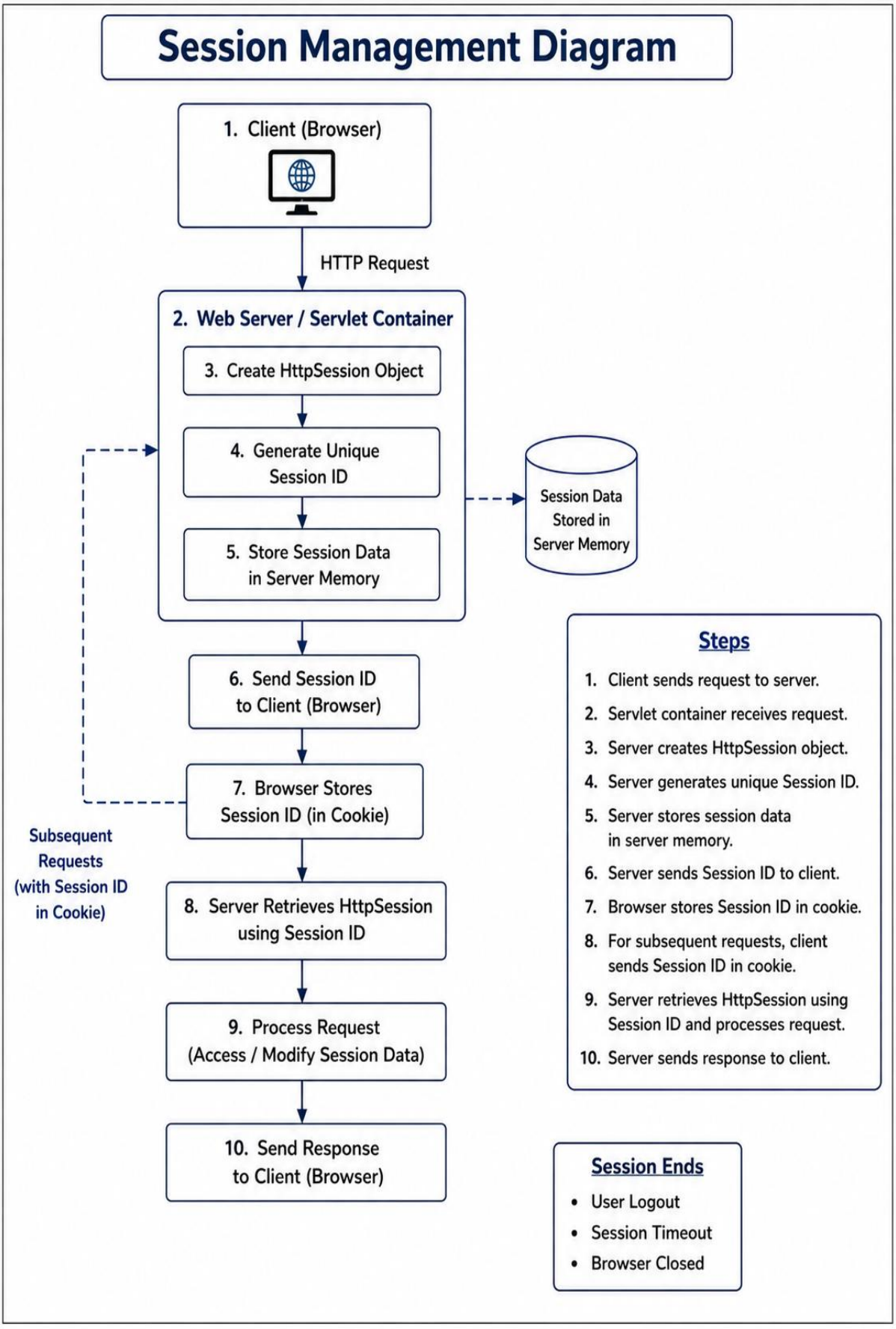
```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/SessionDemo")
public class SessionDemo extends HttpServlet
{
   public void doGet(HttpServletRequest request,
```

```
                HttpServletResponse response)
        throws ServletException, IOException
    {
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession();

        session.setAttribute("username", "Amit");

        String user = (String) session.getAttribute("username");

        out.println("Welcome " + user);
    }
}
```

Output

Welcome Amit

Note: HttpSession is the most secure and widely used session tracking technique in Servlet-based applications because session data is stored on the **server-side**, reducing the risk of unauthorized access compared to client-side techniques such as Cookies.


Session Management Diagram
1. Client (Browser)
HTTP Request
2. Web Server / Servlet Container
3. Create HttpSession Object
4. Generate Unique Session ID
5. Store Session Data in Server Memory
Session Data Stored in Server Memory
6. Send Session ID to Client (Browser)
7. Browser Stores Session ID (in Cookie)
Subsequent Requests (with Session ID in Cookie)
8. Server Retrieves HttpSession using Session ID
9. Process Request (Access / Modify Session Data)
10. Send Response to Client (Browser)
Steps
1. Client sends request to server.
2. Servlet container receives request.
3. Server creates HttpSession object.
4. Server generates unique Session ID.
5. Server stores session data in server memory.
6. Server sends Session ID to client.
7. Browser stores Session ID in cookie.
8. For subsequent requests, client sends Session ID in cookie.
9. Server retrieves HttpSession using Session ID and processes request.
10. Server sends response to client.
Session Ends
• User Logout
• Session Timeout
• Browser Closed

Unit VI: JavaServer Pages (JSP)

6.1 Introduction to JSP

JavaServer Pages (JSP) is a server-side technology used to develop **dynamic and interactive web pages**. It enables developers to embed **Java code** within **HTML** using files with the **.jsp** extension. JSP is built on top of **Servlet technology** and simplifies the development of Java-based web applications by separating the presentation layer from the business logic.

When a client requests a JSP page for the first time, the **JSP Container** translates the JSP page into an equivalent **Servlet**, compiles it into a Java class, loads it into memory, and executes it. A JSP page is translated into a Servlet only during its **first request** or whenever the JSP file is modified. For all subsequent requests, the previously compiled servlet is executed directly, thereby improving the performance of the web application.

JSP provides several built-in features such as **implicit objects, scripting elements, directives, Expression Language (EL), custom tags,** and **JavaBeans** support. These features simplify web application development, reduce coding effort, and improve the readability and maintainability of the application.

JSP is widely used for developing **login pages, registration forms, online examination systems, student management systems, e-commerce websites, online banking applications,** and other **database-driven web applications** where dynamic content is generated according to user requests.

🟦 Features of JSP

- Used to create dynamic web pages.
- Built on top of Servlet technology.
- Uses the **.jsp** file extension.
- Supports embedding Java code within HTML.
- Automatically translated and compiled into a Servlet.
- Provides implicit objects and scripting elements.
- Supports JavaBeans and JDBC integration.
- Improves code readability, reusability, and maintainability.

Note: Every **JSP page** is internally converted into a **Servlet** by the **JSP Container** before execution. Thus, JSP combines the simplicity of HTML with the processing capabilities of Java, making it an efficient technology for developing dynamic web applications.

🟦 6.2 JSP Scripting Elements

JSP Scripting Elements are special tags used to embed **Java code** within a JSP page. They enable developers to generate dynamic content, perform calculations, declare variables and methods, and execute Java statements on the server. JSP provides three scripting elements: **Expression Tag, Scriptlet Tag,** and **Declaration Tag**, each serving a specific purpose during the execution of a JSP page.

When a JSP page is requested, the **JSP Container** translates the JSP page into an equivalent **Servlet**. During this translation process, all JSP scripting elements are converted into the corresponding Java code and become part of the generated servlet.

🟦 1. Expression Tag

The **Expression Tag** is used to display the value of a Java expression directly in the web page. The JSP container evaluates the expression, converts the result into a string, and automatically sends it to the client. It is commonly used to display variables, method return values, and calculated results.

The **Expression Tag** does **not** require a semicolon (;) because it contains a Java expression rather than a Java statement.

Syntax

<%= expression %>

Example

<html>

<body>

Current Date :

<%= new java.util.Date() %>

</body>

</html>

Output

Current Date :

Mon Jul 13 10:30:45 IST 2026

🟦 2. Scriptlet Tag

The **Scriptlet Tag** is used to write Java statements inside a JSP page. The code written within a scriptlet becomes part of the generated servlet's **_jspService()** method. It is mainly used for variable declaration, conditional statements, loops, and other processing logic. The Scriptlet Tag is executed every time a client requests the JSP page.

Syntax

```
<%
   // Java statements
%>
```

Example

```
<html>
<body>
<%
for(int i = 1; i <= 5; i++)
{
   out.println(i + "<br>");
}
%>
</body>
</html>
```

Output

1

2

3

4

5

🟦 3. Declaration Tag

The **Declaration Tag** is used to declare variables, methods, and objects that become members of the generated servlet class. Unlike the Scriptlet Tag, the declarations are placed outside the **_jspService()** method and can be accessed throughout the JSP page. Variables and methods declared using the Declaration Tag become **instance members** of the generated servlet class.

Syntax

```
<%!
   // Variable or method declaration
%>
```

Example

```
<html>
<body>

<%!
int square(int n)
{
   return n * n;
}
%>

Square of 5 = <%= square(5) %>

</body>
</html>
```

Output

Square of 5 = 25

🟦 Difference Between JSP Scripting Elements			
Scripting Element	**Syntax**	**Executed In**	**Purpose**
Expression Tag	<%= ... %>	_jspService()	Displays the value of an expression directly in the output.
Scriptlet Tag	<% ... %>	_jspService()	Executes Java statements such as loops and conditions.
Declaration Tag	<%! ... %>	Servlet Class	Declares variables and methods as members of the generated servlet class.

Note: JSP scripting elements allow Java code to be embedded within a JSP page. The **Expression Tag** is used to display output, the **Scriptlet Tag** is used to execute Java statements, and the **Declaration Tag** is used to declare variables and methods that can be used throughout the JSP page.

🟦 6.3 JSP Directives

JSP Directives are special instructions provided to the **JSP Container** that control how a JSP page is translated into a Servlet. Unlike JSP scripting elements, directives do not generate output directly. Instead, they provide global information about the JSP page, such as importing Java packages, including external files, or declaring custom tag libraries.

JSP directives begin with **<%@** and end with **%>**. They affect the entire JSP page and are processed during the **translation phase**, before the JSP page is compiled into a Servlet. JSP provides three types of directives: **Page Directive, Include Directive,** and **Taglib Directive**.

🟦 Types of JSP Directives

1. Page Directive

The **Page Directive** is used to define page-level settings such as the programming language, imported packages, content type, buffering, session handling, and error pages. These settings apply to the entire JSP page.

Syntax

<%@ page attribute="value" %>

Example

```
<%@ page language="java"
        contentType="text/html"
        import="java.util.Date" %>


<html>
<body>


Today's Date :
<%= new Date() %>


</body>
</html>
```

2. Include Directive

The **Include Directive** is used to include the contents of another file into the current JSP page during the **translation phase**. The included file becomes a part of the generated servlet and is commonly used for reusable components such as **header files, footer files, navigation menus,** and **configuration files**.

Syntax

```
<%@ include file="filename.jsp" %>
```

Example

```
<%@ include file="header.jsp" %>


<h2>Welcome to JSP</h2>


<%@ include file="footer.jsp" %>
```

---

3. Taglib Directive

The **Taglib Directive** is used to declare a custom tag library that can be used within a JSP page. It allows developers to use predefined custom tags instead of writing Java code directly in JSP pages. The most commonly used tag library is the **JavaServer Pages Standard Tag Library (JSTL)**. Tag libraries improve code readability and promote code reusability by replacing Java code with reusable custom tags.

Syntax

```
<%@ taglib uri="URI" prefix="prefixName" %>
```

Example

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core"
        prefix="c" %>


<c:out value="Welcome to JSP" />
```

🟦 Difference Between JSP Directives		
Directive	**Purpose**	**Processed During**
Page Directive	Defines page-level configuration settings.	Translation Phase
Include Directive	Includes another file into the current JSP page.	Translation Phase
Taglib Directive	Declares a custom tag library for use in JSP.	Translation Phase

Note: JSP Directives are processed by the **JSP Container** during the **translation phase**. They provide configuration information for the entire JSP page and do not generate any direct output.

🟦 6.4 Sessions in JSP

Sessions in JSP are used to maintain user-specific information across multiple requests during a user's interaction with a web application. Since **HTTP is a stateless protocol**, it does not remember information about previous requests. JSP overcomes this limitation by using the **HttpSession** interface, which stores user-related information on the **server-side**.

The **session** is one of the **implicit objects** available in JSP. It is automatically created by the **JSP Container** unless it is explicitly disabled using the **page directive** (session="false"). Each user is assigned a unique **Session ID**, which is generally stored in a cookie and sent with every subsequent request. The server uses this Session ID to identify the user and retrieve the corresponding session data.

Sessions in JSP are commonly used to maintain **login information, user preferences, shopping cart details, online examination data, banking transactions,** and other information that must remain available throughout the user's interaction with the application.

🟦 Common Methods of HttpSession	
Method	**Description**
setAttribute(String name, Object value)	Stores an object in the session.
getAttribute(String name)	Retrieves the value of the specified session attribute.
removeAttribute(String name)	Removes a session attribute.
invalidate()	Terminates the current session.
getId()	Returns the unique Session ID.
getCreationTime()	Returns the session creation time.
setMaxInactiveInterval(int interval)	Sets the maximum inactive time (in seconds) before the session expires.

🟦 JSP Example

```
<%@ page session="true" %>
<html>
<body>
<%
session.setAttribute("username", "Amit");
String user = (String) session.getAttribute("username");
%>
<h2>Welcome <%= user %></h2>
</body>
</html>
```

Output

Welcome Amit

🟦 Applications of Sessions in JSP

- User login authentication.
- Online shopping cart management.
- Online examination systems.
- Banking and financial applications.
- User preference management.
- Student management systems.
- Personalized web applications.

Note: In JSP, the **session** is an **implicit object** of type **HttpSession**. It is automatically created by the JSP Container and can be used directly without explicitly creating a HttpSession object. This makes session handling simple, secure, and efficient for maintaining user-specific information across multiple requests.

🟦 6.5 Using JDBC in JSP

JDBC (Java Database Connectivity) is a Java API used to connect Java applications with relational databases such as **MySQL, Oracle, PostgreSQL,** and **SQL Server**. In JSP, JDBC enables web pages to interact with databases for storing, retrieving, updating, and deleting data dynamically. JSP communicates with the database through the **JDBC API**, allowing dynamic retrieval and manipulation of data stored in relational databases.

When a user submits information through an HTML form, the JSP page receives the request, establishes a database connection using the **JDBC Driver** and **DriverManager**, executes the required SQL statement, processes the result, and displays an appropriate response to the user. This enables the development of dynamic, database-driven web applications.

Although database operations can be performed directly in JSP, modern web application development recommends separating database logic from the presentation layer by using **Servlets** or **JavaBeans**. However, using JDBC in JSP is useful for understanding database connectivity and developing small web applications.

🟦 Steps to Use JDBC in JSP

1. Import the required JDBC package.
2. Load the JDBC Driver.
3. Establish a connection with the database.
4. Create a **PreparedStatement** object.
5. Execute the SQL query.
6. Process the query result using **ResultSet**.
7. Close all database resources.

🟦 JSP Example

```
<%@ page import="java.sql.*" %>

<html>
<body>

<%
response.setContentType("text/html");

try
{
   Class.forName("com.mysql.cj.jdbc.Driver");

   Connection con = DriverManager.getConnection(
      "jdbc:mysql://localhost:3306/studentdb",
      "root",
      "password");
```

```
    PreparedStatement ps = con.prepareStatement(
        "SELECT * FROM student");

    ResultSet rs = ps.executeQuery();

    while(rs.next())
    {
%>

ID : <%= rs.getInt("id") %><br>
Name : <%= rs.getString("name") %><br><br>

<%
    }

    rs.close();
    ps.close();
    con.close();
}
catch(Exception e)
{
    out.println(e);
}
%>
```

```
</body>
</html>
```

Output

ID : 1

Name : Amit

ID : 2

Name : Rahul

🟦 Applications of JDBC in JSP

- User registration systems.
- Login authentication.
- Student management systems.
- Employee management systems.
- Online examination systems.
- E-commerce applications.
- Database-driven web applications.

Note: JDBC enables JSP pages to communicate with relational databases. In enterprise-level applications, **database operations should preferably be performed in Servlets or JavaBeans**, while JSP should mainly be used for the presentation layer. This approach improves code maintainability, security, and overall application design.

🟦 6.6 JavaBeans in JSP

JavaBeans are reusable Java classes used to encapsulate data and business logic in Java web applications. A JavaBean is a **serializable Java class** that follows standard programming conventions. It contains a **public no-argument constructor**, **private data members (properties)**, and **public getter and setter methods** to access and modify the properties.

In JSP, JavaBeans are used to separate the **presentation layer** from the **business logic**, making web applications more modular, reusable, and maintainable. JSP pages interact with JavaBeans to store, retrieve, and process data without embedding large amounts of Java code directly in the JSP page.

JavaBeans are widely used in **student management systems, employee management systems, online banking, e-commerce websites, registration systems,** and other **database-driven web applications**.

🟦 Features of JavaBeans

- Reusable Java component.
- Implements the **Serializable** interface.
- Contains private data members (properties).
- Provides public getter and setter methods.
- Includes a public no-argument constructor.
- Supports code reusability and modular programming.
- Separates business logic from the presentation layer.
- Easily integrates with JSP, Servlets, and JDBC.

🟦 JSP Action Tags for JavaBeans	
Action Tag	**Purpose**
<jsp:useBean>	Creates or locates a JavaBean object.
<jsp:setProperty>	Sets the value of a bean property.
<jsp:getProperty>	Retrieves and displays the value of a bean property.

🟦 JavaBean Class

```
import java.io.Serializable;


public class Student implements Serializable
{
   private String name;


   public Student()
   {
   }
   public void setName(String name)
   {
      this.name = name;
   }
   public String getName()
   {
      return name;
   }
}
```

🟦 JSP Example

```
<jsp:useBean id="student" class="Student" />
<jsp:setProperty
   name="student"
   property="name"
   value="Amit" />
Student Name :
<jsp:getProperty
   name="student"
   property="name" />
```

Output

Student Name : Amit

🟦 Applications of JavaBeans

- User registration systems.
- Login authentication.
- Student management systems.
- Employee management systems.
- E-commerce applications.
- Banking applications.
- Database-driven web applications.

Note: A **JavaBean** is a reusable Java class that encapsulates data using **private properties**, **public getter and setter methods**, and a **public no-argument constructor**. It generally implements the **Serializable** interface and is accessed in JSP using the **<jsp:useBean>**, **<jsp:setProperty>**, and **<jsp:getProperty>** action tags to develop modular, reusable, and maintainable web applications.